

Efficient Actively Secure DPF and RAM-based 2PC with One-Bit Leakage

Wenhao Zhang* Xiaojie Guo[†] Kang Yang[‡] Ruiyu Zhu[§] Yu Yu[¶] Xiao Wang^{||}

March 12, 2024

Abstract

Secure two-party computation (2PC) in the RAM model has attracted huge attention in recent years. Most existing results only support semi-honest security, with the exception of Keller and Yanai (Eurocrypt 2018) with very high cost. In this paper, we propose an efficient RAM-based 2PC protocol with active security and one-bit leakage.

1. We propose an actively secure protocol for distributed point function (DPF), with one-bit leakage, that is essentially as efficient as the state-of-the-art semi-honest protocol. Compared with previous work, our protocol takes about $50\times$ less communication for a domain with 2^{20} entries, and no longer requires actively secure generic 2PC.
2. We extend the dual-execution protocol to allow reactive computation, and then build a RAM-based 2PC protocol with active security on top of our new building blocks. The protocol follows the paradigm of Doerner and Shelat (CCS 2017). We are able to prove that the protocol has end-to-end one-bit leakage.
3. Our implementation shows that our protocol is almost as efficient as the state-of-the-art semi-honest RAM-based 2PC protocol, and is at least two orders of magnitude faster than prior actively secure RAM-based 2PC without leakage, providing a realistic trade-off in practice.

1 Introduction

Secure two-party computation (2PC) protocols [Yao86] allow two parties each with a private input x, y respectively, to obtain $f(x, y)$ for some public function f but nothing else. There has been a huge amount of work to build efficient protocols and tools when f can be efficiently represented as a *circuit*; however, not all functions can be converted to a compact circuit since normal programs are in the *random-access machine (RAM) model*. To address this, secure 2PC in the RAM model [GKK⁺12] was proposed to support private accesses in 2PC protocols. It has found a lot of applications for building efficient and secure protocols for database queries [BEE⁺17], stable matching [DEs16] and various graph algorithms [LWN⁺15].

The high-level approach of RAM-based 2PC is to combine oblivious RAMs (ORAMs) [GO96] and 2PC protocols. In more detail, one can use a 2PC protocol to emulate an ORAM client securely while having the parties act as the ORAM server(s): since the ORAM ensures that the server does not learn the private accesses, the parties cannot learn the accesses either. Although there has been huge progress in pushing the efficiency of RAM-based 2PC by means of optimized ORAM for secure computation [GGH⁺13,

*Northwestern University, wenhao.zhang@northwestern.edu

[†]Nankai University & State Key Laboratory of Cryptology, xiaojie.guo@mail.nankai.edu.cn

[‡]State Key Laboratory of Cryptology, yangk@sklc.org

[§]rynzhu@gmail.com

[¶]Shanghai Jiao Tong University & Shanghai Qi Zhi Institute, yuyu@yuyu.hk

^{||}Northwestern University, wangxiao@northwestern.edu

GKK⁺12, KS14, WHC⁺14, WCS15, ZWR⁺16] and customized protocols leveraging the fact that there are two non-colluding ORAM servers with computational resources [GKW18, AFN⁺17, Ds17, FJKW15, LO13, VH23, HV21], all of them are in the semi-honest setting. The only exception is the work of Keller and Yanai [KY18] (dubbed KY18), where they proposed an optimized protocol based on the Circuit ORAM [WCS15] and the SPDZ-BMR protocol [LPSY15]. When comparing its performance with state-of-the-art semi-honest protocols [Ds17], we observe a huge gap of at least two orders of magnitude slowdowns, making it essentially infeasible to run any RAM-based 2PC applications in the malicious setting. When diving into the details, there are two main sources of slowdown.

1. **Actively secure circuit-based 2PC has a high overhead.** The generic approach of RAM-based 2PC can be done with malicious security by emulating the ORAM client in a reactive 2PC with malicious security. Indeed, this is the approach that KY18 took. However, due to the high depth of circuits needed to emulate ORAM circuits, a constant-round malicious 2PC is the only option. KY18 used the SPDZ-BMR protocol, which allows identification in the event of abort; this feature is crucial to enable their efficient representation of the server verifiable secret sharing, which can lead to two orders of magnitude improvements in memory usage. KY18 also posted an open problem on how to make it compatible with more efficient authenticated garbling [WRK17a] approach, which is still open to this date. Regardless, constant-round maliciously secure 2PC generally incurs a significant performance slowdown and this overhead will be amplified in a RAM protocol when emulating the ORAM algorithm in 2PC.
2. **Tricks in semi-honest protocols no longer work directly.** State-of-the-art RAM-based 2PC protocols use a crucial tool, namely distributed point function (DPF) [GI14, BGI16], which allows two parties with secret shares of α and β to homomorphically evaluate the point function $f_{(\alpha, \beta)}(x)$, that evaluates to β only when x equals α and 0 otherwise; recent DPFs [Ds17, GYW⁺23] let parties obtain secret shares of the output with communication sublinear to the number of evaluations. This implies an efficient protocol to read or write an array but not both at the same time. Doerner and Shelat [Ds17] first proposed a protocol, namely Floram, using DPF on top of the square-root ORAM, which was later improved in a sequence of works [HV21, VH23]. However, bringing the same trick to malicious security is challenging: 1) it is not clear how to efficiently distribute DPF keys based on shares of α and β with malicious security; 2) it is unclear how to ensure the correctness of the local computation, an important feature of DPF-based ORAMs.

Contribution. In this paper, we design and implement a maliciously secure RAM-based 2PC protocol with high concrete performance. The protocol would leak one bit of information to the adversary but enjoys performance essentially the same as state-of-the-art semi-honest RAM 2PC protocols.

1. We design an efficient and maliciously secure protocol for distributed point functions (DPFs). Compared to previous malicious protocols, our protocol follows a different route in generating the DPF correlation and no longer needs generic malicious 2PC. As a result, our protocol improves the communication by a factor of $50\times$. What's more, the cost of this protocol is almost the same as the state-of-the-art semi-honest DPF protocols [GYW⁺23]. It also has huge applications beyond RAM-based 2PC, e.g., in malicious pseudorandom correlation generators.
2. We extend the normal dual-execution with one-bit leakage protocol [HKE12] to support reactive 2PC. Then we incorporate both building blocks to build a malicious RAM-based 2PC based on the blueprint of Floram. Although DPF is invoked repeatedly, we show an optimization that allows end-to-end leakage to be a single bit by carefully controlling the abort event.
3. We implement all of the protocols and hook them with generic malicious 2PC for end-to-end applications. Our benchmark shows that the performance of our active-secure one-bit leakage protocol is almost as fast

as semi-honest protocols in common network settings and is two orders of magnitude faster than prior full malicious RAM-based 2PC [KY18].

Paper organization. Section 2 provides an overview of our techniques and improvements. In Section 3, we introduce preliminaries. In Section 4, we provide details of our reactive 2PC protocol; in Section 5, we show our efficient DPF protocol details. We combine them together to build a RAM-based 2PC protocol in Section 6. Finally, in Section 7, we discuss the concrete performance of our protocols.

2 Technical Overview

2.1 Recap of Floram

First, we review the high-level ideas of Floram [Ds17], one of the state-of-the-art semi-honest RAM-based 2PC protocols. The protocol has a read-only memory (ROM), a write-only memory (WOM), and a stash (S) supporting both read and write. Suppose that the initial values are in both the ROM and WOM; the protocol will ensure that 1) WOM always contains the most recent data (but we cannot read from it) and 2) ROM and S as a whole also contain the most recent data where the version in S takes priority. For a read operation, one just needs to query from the ROM structure and then linearly scan all elements in S; for a write operation, one first updates the WOM, and then appends this update to S. Both ROM and WOM can be efficiently built using DPFs. When S reaches σ elements, a refresh protocol will be executed that copies over the data in WOM to ROM and clears the stash S. Due to the advances in DPF, the communication cost of an operation on ROM and WOM is $O(\log N)$ for an array of size N ; the stash is instead implemented using generic 2PC protocols. Thus the amortized communication cost is $O(\log N + \sigma + N/\sigma)$, which minimizes to $O(\sqrt{N})$.

In order to bring this idea to malicious security, we need to make all building blocks maliciously secure and allow them to be composed without causing inconsistency. Below, we discuss the details of each component.

2.2 Reactive 2PC with One-Bit Leakage

Next, we briefly discuss the intuition in our reactive 2PC protocol. Active 2PC with one-bit leakage was studied before [HKE12, MF06], but was only assumed as two parties evaluate a function for one shot. Their intuition is to run Yao’s garbled circuit protocol twice with opposite directions along with malicious oblivious transfer and run a check protocol in the end to ensure the consistency of two executions. Either the output is correct, or the protocol will abort; thus the adversary can only learn one bit of information from the fact that the protocol aborts or proceeds. However, in our setting, two parties need to hold a “state” (e.g., stash) that is fed to a reactive 2PC and gets updated by the protocol.

To enable this upgrade, we hook the idea of dual execution with BDOZ authenticated shares [BDOZ11]. To authenticate a secret sharing of a bit b as BDOZ share (namely $\langle\langle b \rangle\rangle$), party P_0 holds $(b_0, M_0[b_0], K_0[b_1])$ and P_1 holds $(b_1, M_1[b_1], K_1[b_0])$, such that $M_0[b_0] = K_1[b_0] \oplus b_0\Delta_1$ and $M_1[b_1] = K_0[b_1] \oplus b_1\Delta_0$ where Δ_0, Δ_1 are private MAC keys held by P_0 and P_1 respectively. When P_0 is the garbler, we let it produce a garbled circuit (GC) where the free-XOR delta is Δ_0 . For an input bit b in BDOZ share, P_0 can define a zero garbled key as $L^0 = K_0[b_1] \oplus b_0\Delta_0$ and P_1 defines $L^* = M_1[b_1]$, we can see that

$$L^* = M_1[b_1] = K_0[b_1] \oplus (b_0 \oplus b) \cdot \Delta_0 = L^0 \oplus b\Delta_0.$$

This means that L^* held by P_1 as an evaluator and L^0 held by P_0 as a garbler have a correct relationship needed for GC generation/evaluation. In summary, this is an approach where two parties can *locally* convert BDOZ shares to garbled labels compatible with dual execution. There is a similar process making dual-execution garbled labels back to BDOZ shares *locally*, although the shares may not be valid if one of the parties cheats during GC execution. To obtain the output with guaranteed correctness, two parties need first to check the validity of the authenticated share and only reveal it if it is valid. One bit of leakage is due to the validity check.

With this intuition, two parties store any state in BDOZ shares and convert them to garbled labels when they need to run 2PC, where the results can be converted back to BDOZ shares. The overhead compared to semi-honest Yao’s protocol is exactly twice, but it can be parallelized easily.

2.3 Efficient DPF with Malicious Security

Prior protocol. Recall that in a DPF protocol, two parties have secret sharing of $\alpha \in [n]$ and $\beta \in \mathbb{F}$ and should get secret sharings of a size- n vector $x \in \mathbb{F}^n$, which is all zeros except that $x^{(\alpha)} = \beta$. To make the DPF protocol maliciously secure, there are two important tasks: 1) use authenticated sharing for the input and output of the DPF protocol, and 2) prevent the parties from cheating during the execution of the protocol. Ensuring DPF to output authenticated sharing can be done via appending β with $\beta \cdot \Delta$, where Δ is the secret shared MAC key; this works as long as the DPF scheme allows any ring element as β . However, ensuring input authentication, consistency, and protocol security is much more complicated, as the state-of-the-art DPF protocol involves $\log N$ rounds and extensive local computation. The only maliciously secure protocol was proposed by Boyle et al. [BCG⁺20]. Their protocol works by first generating additive shares of vector in the form of $([0], \dots, [0], [r], [0], \dots, [0])$, where the share of a random value r is in the α -th location, following the classical semi-honest DPF protocol but replace all joint computation using a generic malicious 2PC. Then, two parties further expand a level to obtain shares of $2n$ elements: $([0], \dots, [0], [L], [R], [0], \dots, [0])$, where L, R are random values and $[L]$ is the 2α -th element. Two parties then again use generic malicious 2PC to compute authenticated shares of L^{-1} and R^{-1} while only revealing R^{-1} . Next, two parties pick a public random value χ and compute two linear combinations on their secret sharings which will end up being $X_L = \chi^\alpha \cdot L$ and $X_R = \chi^\alpha \cdot R$. Finally, they can check whether $[X_L] \cdot [L^{-1}] = [X_R] \cdot R^{-1}$ in malicious 2PC.

Their analysis shows that this protocol unfortunately leaks one bit of information about α to the adversary. In terms of the cost, this protocol requires heavy use of generic malicious 2PC and, in particular, needs to compute three field multiplications in 2PC, which is very expensive. For example, MASCOT [KOS16] requires about 33,000 bytes of communication to compute one such multiplication even without counting the cost of underlying oblivious transfer, while the rest part of this protocol only needs $2(\log N + 1)\kappa$ bits of communication. This means that the cost of this field multiplication is going to be the main bottleneck of the whole DPF protocol for any reasonable size of N . Another potential issue is that this malicious DPF protocol requires β to be a field element (so that inverse exists), and thus it is not immediately clear how to efficiently support output authentication, where β has two field elements.

Our protocol. The prior protocol is costly and also heavily relies on malicious generic 2PC, making it complicated to implement. In this work, we propose a completely different way to generate DPF with malicious security without using generic malicious 2PC or field multiplication, while still maintaining the same level of security. As a result, the protocol is much easier to implement and is almost as efficient as state-of-the-art semi-honest DPF protocols.

Different from the prior work that first generates the whole vector of shares and then checks the relationship in a modular way, our protocol maintains the invariance that after each level of expansion, two parties hold authenticated sharing of partial prefix expansion. To be more specific, we assume that the two parties start with a SPDZ authenticated share of 1, namely $\llbracket 1 \rrbracket$. A SPDZ authentication is similar to BDOZ but instead parties hold secret shares of the value b and its MAC $b \cdot \Delta$ (along with secret shares of the MAC key), i.e., (b_0, M_0) and (b_1, M_1) such that $M_1 \oplus M_0 = (b_0 \oplus b_1) \cdot (\Delta_0 \oplus \Delta_1)$. See Section 3.3 for complete details. Two parties use one level of expansion to either get $(\llbracket 0 \rrbracket, \llbracket 1 \rrbracket)$ or $(\llbracket 1 \rrbracket, \llbracket 0 \rrbracket)$, depending on the most significant bit of α . This process can be iteratively executed to obtain $(\llbracket 0 \rrbracket, \dots, \llbracket 0 \rrbracket, \llbracket 1 \rrbracket, \llbracket 0 \rrbracket, \dots, \llbracket 0 \rrbracket)$, where $\llbracket 1 \rrbracket$ is at the α -th location. Finally, a correction word is used to correct $\llbracket 1 \rrbracket$ to $\llbracket \beta \rrbracket$ while maintaining $\llbracket 0 \rrbracket$ unchanged.

Given this high-level approach, the key is to expand one level of the tree. Our high-level idea follows a semi-honest optimization of DPF, namely Half-Tree [GYW⁺23]. Suppose two parties hold (x_0, X_0) and

(x_1, X_1) respectively as their SPDZ share of 1 at the root such that $X_0 \oplus X_1 = \Delta_0 \oplus \Delta_1$. To obtain $(\llbracket a \rrbracket, \llbracket a \oplus 1 \rrbracket)$ for some private $a \in \{0, 1\}$, with a hash function $\mathcal{H}$ the correction word CW would be

$$\text{CW} := \mathcal{H}(x_0 \| X_0) \oplus \mathcal{H}(x_1 \| X_1) \oplus (a \oplus 1) \cdot (\Delta_0 \oplus \Delta_1).$$

Each party can locally expand the left-child node as $(l_b, L_b) := \mathcal{H}(x_b \| X_b) \oplus x_b \cdot \text{CW}$ and the right-child node as $(r_b, R_b) := \mathcal{H}(x_b \| X_b) \oplus X_b \oplus x_b \cdot \text{CW}$. In addition, computing CW boils down to compute $a \cdot \Delta$ efficiently; when a is authenticated, their shares can be used to reconstruct shares of $a \cdot \Delta$ locally; thus computing shares of CW can all be done via local computation. Our crucial observation is that, the adversary can only cheat by corrupting CW with an additive value. However, if such corruption happens, the only type of change is to make the authenticated shares on the next level $((l_b, L_b), (r_b, R_b))$ in the above example) invalid, which can be easily discovered by an almost-free MAC check protocol. Unfortunately, the adversary can still learn one-bit information since its cheat could lead to an abort event or not, depending on the bit a . However, it is sufficient in our application and many other applications in pseudorandom correlation generators. We refer to Section 5 for more details.

2.4 Putting Everything Together

Given the above two important building blocks already optimized with high efficiency, we can now build an efficient RAM-based 2PC protocol with active security. We follow the blueprint of Floram [Ds17] and use authenticated shares, either in BDOZ or SPDZ, to connect various building blocks. Here the main challenge is to avoid secure computation of pseudorandom functions (PRFs) during refresh protocols, which would be prohibitive. It is clear that for WOM, two parties would store the authenticated shares, but the design of ROM is more complicated (as we elaborate below). Our final solution in the end only requires 2 PRF computations in 2PC for each operation.

Write-only memory. Suppose elements stored in the RAM model are represented as an array D with totally N elements. For WOM, two parties need to hold authenticated shares of $D^{(i)}$. To update the α -th value to D^* , two parties first read from ROM to obtain $\llbracket D^{(\alpha)} \rrbracket$ and then use DPF to obtain an authenticated vector of field elements $(\dots, \llbracket 0 \rrbracket, \llbracket D^{(\alpha)} \oplus D^* \rrbracket, \llbracket 0 \rrbracket, \dots)$, where the non-zero element is at location α , and then locally XOR each element in the list to the authenticated shares of $D^{(0)}, \dots, D^{(N-1)}$ corresponding. Although this version requires two separate DPFs, one can apply the optimization in Floram to reduce it to call the DPF protocol only once. We provide full details in Section 6.

Read-only memory: First attempt. Two parties hold authenticated sharing of two PRF keys $\llbracket k_0 \rrbracket$ and $\llbracket k_1 \rrbracket$. For ROM, we can think of a scheme where the i -th data block $D^{(i)}$ is encrypted as $E^{(i)} = \text{PRF}(k_0, i) \oplus \text{PRF}(k_1, i) \oplus D^{(i)}$ and is public to both parties. For a read operation at α , two parties would use the above malicious DPF protocol to obtain a unit vector $(\dots, \llbracket 0 \rrbracket, \llbracket 1 \rrbracket, \llbracket 0 \rrbracket, \dots)$, where $\llbracket 1 \rrbracket$ is specified by α . Then two parties can compute $\llbracket E^{(\alpha)} \rrbracket$ by computing the inner product between the vector E and the authenticated unit vector, and then use 2PC to decrypt it to obtain the authenticated share $\llbracket D^{(\alpha)} \rrbracket$. So far, everything works great, but the challenge appears when connecting WOM to ROM via a refresh procedure. Essentially, the problem setup is that two parties have $\llbracket k_0 \rrbracket, \llbracket k_1 \rrbracket$ and $\llbracket D^{(i)} \rrbracket$; we need a protocol so that they obtain $E^{(i)} = \text{PRF}(k_0, i) \oplus \text{PRF}(k_1, i) \oplus D^{(i)}$. To defend against a malicious adversary, the values held by the honest party should be correct even if the adversary cheats in some way. One way to ensure this property is to mask all PRFs in a 2PC protocol, but this would require $2N$ PRF computation in 2PC. This computation would blow up the cost since it can only cover about $\sqrt{N}$ writes efficiently, leading to perform PRF computation $O(\sqrt{N})$ times in 2PC per access. Alternatively, two parties can compute PRF locally, supply them to 2PC to compute the masking step, and then reveal the result; however, this approach would allow parties to change the value as the adversary can claim any value as their PRF evaluation. In summary, it is not clear how to ensure consistency between WOM and ROM.

Read-only memory: Our approach. Our alternative method is to put the value and its SPDZ MAC together into the ROM. Since data are doubly encrypted by both parties, no information can be revealed.

Furthermore, the additional MAC allows us to ensure consistency. In more detail, we now have $E^{(i)} = \text{PRF}(k_0, i) \oplus \text{PRF}(k_1, i) \oplus (D^{(i)}, D^{(i)} \cdot \Delta)$, where $\Delta = \Delta_0 \oplus \Delta_1$ is the SPDZ MAC key and the output length of PRF is sufficient for two elements. To read the α -th element from the array, two parties first compute $[\text{PRF}(k_0, \alpha) \oplus \text{PRF}(k_1, \alpha)]$ in 2PC, and use malicious DPF to obtain XOR-secret sharing of a unit vector $[u] = (\dots, [0], [1], [0], \dots)$, where the non-zero value is at index α . Two parties locally compute $(\bigoplus_i [u^{(i)}] \cdot E^{(i)}) \oplus [\text{PRF}(k_0, \alpha) \oplus \text{PRF}(k_1, \alpha)] = [(D^{(\alpha)}, D^{(\alpha)} \cdot \Delta)]$, which is essentially the SPDZ authenticated sharing $\llbracket D^{(\alpha)} \rrbracket$. Here we no longer need MACs on the output of DPF as long as DPF is maliciously secure: if any party cheats in any way, SPDZ shares as the output will be invalid independent of the underlying data.

Back to refresh procedure: now two parties have sharings $\llbracket k_0 \rrbracket, \llbracket k_1 \rrbracket, \llbracket D^{(i)} \rrbracket$ and need to obtain $E^{(i)} = \text{PRF}(k_0, i) \oplus \text{PRF}(k_1, i) \oplus (D^{(i)}, D^{(i)} \cdot \Delta)$. Two parties can treat the SPDZ sharing $\llbracket D^{(i)} \rrbracket$ as additive sharing $[(D^{(i)}, D^{(i)} \cdot \Delta)]$. Since P_0 can compute $\text{PRF}(k_0, i)$ while P_1 can compute $\text{PRF}(k_1, i)$, they effectively have additive shares $[\text{PRF}(k_0, i) \oplus \text{PRF}(k_1, i) \oplus (D^{(i)}, D^{(i)} \cdot \Delta)]$. To reveal the underlying value, we can just allow them to exchange the shares. Since the public values themselves will eventually be used as authenticated values, any change of values will cause abort.

Bounding the leakage. With the above changes, the protocol is essentially as cheap as its semi-honest counterpart. However, a naive argument would lead to an amount of leakage linear to the number of RAM access operations, since every operation requires outputting some value, where checks are needed, leaving an opportunity to leak a bit. To reduce the amount of leakage, we batch all checks since they all verify consistency between values and their MACs, and defer these checks right before revealing the designated output (i.e., $f(x, y)$ where f is the function in the RAM model to be evaluated). For any intermediate values, we will open them without a check. This will not leak any information because all opened intermediate values are masked by authenticated shares of random values as how we design the protocol. This way, all intermediate values can be simulated while the only abort end is in the end.

3 Preliminaries

3.1 Notation

We use λ to denote the computational security parameter. We denote by $\log(\cdot)$ the logarithm in base 2. We write $x \leftarrow S$ to denote sampling x uniformly at random from a set S . We define $[a, b) := \{a, \dots, b-1\}$ and $[a, b] := \{a, \dots, b\}$. For an n -bit integer x , we denote by $(x^{(0)}, \dots, x^{(n-1)})$ its bit decomposition, that is, $x^{(i)} \in \{0, 1\}$ for $i \in [0, n)$ and $x = \sum_{i \in [0, n)} x^{(i)} \cdot 2^i$. We use bold lower-case letters like $\mathbf{x}$ to denote a vector and $x^{(i)}$ to denote the i -th component of $\mathbf{x}$ with $x^{(0)}$ the first component. We use $\text{lsb}(x)$ to denote the least significant bit (LSB) of a string x (i.e., $x^{(0)}$). We write $\mathbb{F}_{2^\lambda} \cong \mathbb{F}_2[X]/f(X)$ for a monic irreducible polynomial $f(X)$ of degree λ . We use $\mathbf{X} \in \mathbb{F}_{2^\lambda}$ to denote the element corresponding to $X \in \mathbb{F}_2[X]/f(X)$. Depending on the context, we use $\{0, 1\}^\lambda$, $\mathbb{F}_2^\lambda$ and $\mathbb{F}_{2^\lambda}$ interchangeably, and thus addition in $\mathbb{F}_2^\lambda$ and $\mathbb{F}_{2^\lambda}$ corresponds to XOR in $\{0, 1\}^\lambda$. We use $\mathbf{unit}(N, \alpha) \in \mathbb{F}_{2^\lambda}^N$ for a vector with exact one non-zero entry 1 at position $\alpha \in [0, N)$.

3.2 Security Model and Ideal Functionalities

We use the standard ideal/real paradigm [Can00, Gol04] to prove security of our two-party protocols in the presence of a *malicious, static* adversary. In the *ideal-world* execution, two parties P_0 and P_1 interact with an ideal functionality $\mathcal{F}$, and one of them may be corrupted by an *ideal-world adversary* (a.k.a., *simulator*) $\mathcal{S}$. In the *real-world* execution, P_0 interacts with P_1 via executing a protocol Π , and one of them may be corrupted by a *real-world adversary* $\mathcal{A}$. We say that a protocol Π securely realizes an ideal functionality $\mathcal{F}$, if the real-world execution is computationally indistinguishable from the ideal-world execution.

Our protocols call the standard two-party functionalities: the coin-tossing functionality $\mathcal{F}_{\text{coin}}$ and the commitment functionality $\mathcal{F}_{\text{com}}$, which can be securely realized using a random oracle [DKL⁺13].

Functionality $\mathcal{F}_{\text{aBit}}$

Initialize: This command is called only once. Upon receiving (init, Δ_0) from P_0 and (init, Δ_1) from P_1 , where $\Delta_0, \Delta_1 \in \mathbb{F}_{2^\lambda}$, abort if $\text{lsb}(\Delta_0 \oplus \Delta_1) \neq 1$; otherwise store (Δ_0, Δ_1) .

Authenticate bits: This command can be called multiple times. For $b \in \{0, 1\}$, upon receiving $(\text{auth}, b, \mathbf{x}, \ell)$ from P_b and (auth, b, ℓ) from P_{1-b} , where $\mathbf{x} \in \mathbb{F}_{2^\lambda}^\ell$, do the following:

- Sample $K_{1-b}[\mathbf{x}] \leftarrow \mathbb{F}_{2^\lambda}^\ell$. If P_{1-b} is corrupted, instead receive $K_{1-b}[\mathbf{x}] \in \mathbb{F}_{2^\lambda}^\ell$ from the adversary.
- Compute $M_b[\mathbf{x}] := K_{1-b}[\mathbf{x}] + \mathbf{x} \cdot \Delta_{1-b} \in \mathbb{F}_{2^\lambda}^\ell$. If P_b is corrupted, receive $M_b[\mathbf{x}] \in \mathbb{F}_{2^\lambda}^\ell$ from the adversary, and then recompute $K_{1-b}[\mathbf{x}] := M_b[\mathbf{x}] + \mathbf{x} \cdot \Delta_{1-b} \in \mathbb{F}_{2^\lambda}^\ell$.
- Output $M_b[\mathbf{x}]$ to P_b and $K_{1-b}[\mathbf{x}]$ to P_{1-b} .

Figure 1: Functionality for authenticated bits.

3.3 Authenticated Secret Sharings

We consider two kinds of authenticated secret sharings in the two-party setting, i.e., SPDZ style [DPSZ12, DKL⁺13] and BDOZ style [BDOZ11]. Suppose that P_0 (resp., P_1) holds a global key $\Delta_0 \in \mathbb{F}_{2^\lambda}$ (resp., $\Delta_1 \in \mathbb{F}_{2^\lambda}$).

We use $\llbracket x \rrbracket$ to denote a SPDZ-style authenticated secret sharing on $x \in \mathbb{F}_{2^\lambda}$. In particular, we have $\llbracket x \rrbracket = (\llbracket x \rrbracket_0, \llbracket x \rrbracket_1)$ and, for each $b \in \{0, 1\}$, P_b holds

$$\llbracket x \rrbracket_b := (x_b, M_b[x]) \in \mathbb{F}_{2^\lambda}^2$$

such that $x = x_0 + x_1$ and $M_0[x] + M_1[x] = x \cdot (\Delta_0 + \Delta_1) \in \mathbb{F}_{2^\lambda}$. We use $[x] = ([x]_0, [x]_1)$ to denote an unauthenticated additive sharing, i.e., $[x]_0 + [x]_1 = x$. So, we have $\llbracket x \rrbracket = ([x], [x \cdot \Delta])$ with $\Delta = \Delta_0 + \Delta_1$. Note that SPDZ-style authenticated sharings are additively homomorphic, i.e., two parties can locally compute $\llbracket a \cdot x + b \cdot y \rrbracket = a \cdot \llbracket x \rrbracket + b \cdot \llbracket y \rrbracket$ for any public constants $a, b \in \mathbb{F}_{2^\lambda}$. Besides, for any public constant c , both parties can locally compute $\llbracket c \rrbracket$ by setting $x_0 := c$, $x_1 := 0$, $M_0[x] := c \cdot \Delta_0$ and $M_1[x] := c \cdot \Delta_1$. For a vector $\mathbf{x} \in \mathbb{F}_{2^\lambda}^\ell$, we write $\llbracket \mathbf{x} \rrbracket = (\llbracket x^{(0)} \rrbracket, \dots, \llbracket x^{(\ell-1)} \rrbracket)$. In Figure 2, we describe the batch-check protocol with essentially no communication, which can verify the correctness of multiple values opened in a batch.

For a bit $x \in \mathbb{F}_2$, we write $\langle\langle x \rangle\rangle$ to denote a BDOZ-style authenticated secret sharing. In particular, we have $\langle\langle x \rangle\rangle := (\langle\langle x \rangle\rangle_0, \langle\langle x \rangle\rangle_1)$ and, for each $b \in \{0, 1\}$, P_b holds

$$\langle\langle x \rangle\rangle_b = (x_b, K_b[x_{1-b}], M_b[x_b]) \in \mathbb{F}_2 \times \mathbb{F}_{2^\lambda}^2$$

such that secret bit $x = x_0 \oplus x_1$ and MAC tag $M_b[x_b] = K_{1-b}[x_b] + x_b \cdot \Delta_{1-b} \in \mathbb{F}_{2^\lambda}$. The BDOZ-style authenticated sharings can be generated by calling the functionality $\mathcal{F}_{\text{aBit}}$ (shown in Figure 1). In this figure, for the sake of simplicity, we write $\mathbf{x} = (x^{(0)}, \dots, x^{(\ell-1)})$, $K_{1-b}[\mathbf{x}] = (K_{1-b}[x^{(0)}], \dots, K_{1-b}[x^{(\ell-1)}])$ and $M_b[\mathbf{x}] = (M_b[x^{(0)}], \dots, M_b[x^{(\ell-1)}])$. This functionality has been used in previous works [WRK17a, WRK17b, HSS17, YWZ20]. Functionality $\mathcal{F}_{\text{aBit}}$ can be securely realized against malicious adversaries by executing a correlated oblivious transfer (COT) protocol [KOS15, BCG⁺19, YWL⁺20, WYKW21, Roy22, BCG⁺22, GYW⁺23]. To guarantee $\text{lsb}(\Delta_0 \oplus \Delta_1) = 1$, the consistency check in [CWYY23] can be adopted (particularly, λ random authenticated sharings need to be sacrificed). It is clear that BDOZ-style authenticated sharings are also additively homomorphic. For a bit vector $\mathbf{x} \in \mathbb{F}_2^\ell$, we write $\langle\langle \mathbf{x} \rangle\rangle = (\langle\langle x^{(0)} \rangle\rangle, \langle\langle x^{(1)} \rangle\rangle, \dots, \langle\langle x^{(\ell-1)} \rangle\rangle)$.

Both parties can locally compute an authenticated sharing on a field element $x \in \mathbb{F}_{2^\lambda}$ from λ authenticated sharings $\langle\langle x^{(0)} \rangle\rangle, \dots, \langle\langle x^{(\lambda-1)} \rangle\rangle$ where $x^{(i)} \in \{0, 1\}$ for each $i \in [0, \lambda)$. In particular, both parties are able to locally compute $\langle\langle x \rangle\rangle := \sum_{i \in [0, \lambda)} \langle\langle x^{(i)} \rangle\rangle \cdot X^i$. We denote by $\langle\langle x \rangle\rangle := \text{B2F}(\langle\langle x^{(0)} \rangle\rangle, \dots, \langle\langle x^{(\lambda-1)} \rangle\rangle)$

Protocol $\Pi_{\text{BatchCheck}}$

Input: Two parties P_0 and P_1 hold ℓ SPDZ-style authenticated sharings $\llbracket y^{(0)} \rrbracket, \dots, \llbracket y^{(\ell-1)} \rrbracket$ along with their opened values $y^{(i)} \in \mathbb{F}_{2^\lambda}$ for each $i \in [0, \ell)$.

Batch check: Two parties do the following.

1. Two parties call $\mathcal{F}_{\text{coin}}$ to sample a random $\chi \in \mathbb{F}_{2^\lambda}$.
2. Two parties locally compute $\llbracket z \rrbracket := \sum_{i \in [0, \ell)} \chi^i \cdot \llbracket y^{(i)} \rrbracket$ and $z := \sum_{i \in [0, \ell)} \chi^i \cdot y^{(i)} \in \mathbb{F}_{2^\lambda}$.
3. For each $b \in \mathbb{F}_2$, P_b computes $V_b := M_b[z] + z \cdot \Delta_b$ and calls $\mathcal{F}_{\text{com}}$ to commit to V_b .
4. For each $b \in \mathbb{F}_2$, P_b calls $\mathcal{F}_{\text{com}}$ to open V_b . Then, two parties check $V_0 = V_1$ and abort if the check fails.

Figure 2: Protocol for batch-checking the values authenticated by SPDZ-style MACs in the $(\mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.

this local computation. Besides, we can transform a BDOZ-style authenticated sharing to a SPDZ-style authenticated sharing without any interaction [BLN⁺21]. Specifically, given $\langle\langle x \rangle\rangle = (\langle\langle x \rangle\rangle_0, \langle\langle x \rangle\rangle_1)$, both parties locally compute $\llbracket x \rrbracket$ by setting $\llbracket x \rrbracket_b := (x_b, K_b[x_{1-b}] \oplus M_b[x_b] \oplus x_b \Delta_b)$ for each $b \in \{0, 1\}$. We write $\llbracket x \rrbracket := \text{Convert}(\langle\langle x \rangle\rangle)$ for this computation.

3.4 Garbling Scheme

Following the previous work [BHR12], we give the definition of garbling schemes, which is specified for our usage. For a bit $x \in \{0, 1\}$, we use $K[x] \in \{0, 1\}^\lambda$ to denote the 0-label and $M[x] \in \{0, 1\}^\lambda$ to denote the garbled label on bit x . We always consider that the free-XOR technique [KS08] is adopted, which is the case for the state-of-the-art garbling schemes [ZRE15, RR21]. In this case, a random global key $\Delta \in \{0, 1\}^\lambda$ is sampled, and $M[x] = K[x] \oplus x\Delta$ for any bit $x \in \{0, 1\}$. We observe that garbled labels have the same form of BDOZ-style authenticated bits (modeled in functionality $\mathcal{F}_{\text{aBit}}$). In our 2PC protocol shown in Section 4, we will call functionality $\mathcal{F}_{\text{aBit}}$ to generate garbled labels on input wires. Thus, Δ and 0-labels corresponding to input bits have been defined by the BDOZ-style authenticated bits, and are able to be used as the input of garbling algorithm Garble. Similarly, the garbled labels on input bits are defined by the MAC tags in the authenticated bits, and can be used as the input of evaluation algorithm Eval. We will transform the garbled labels on output bits into authenticated bits, instead of decoding them to obtain the output bits. Overall, our 2PC protocol only needs two algorithms Garble and Eval, where the encoding and decoding algorithms are not required.

Definition 1. A garbling scheme $\mathcal{GS} = (\text{Garble}, \text{Eval})$, which is specific to our application, consists of the following two algorithms.

- $(GC, K[y]) \leftarrow \text{Garble}(K[x], \Delta, \mathcal{C})$: Given a vector of 0-labels $K[x]$ on input wires, a global key Δ and a Boolean circuit $\mathcal{C} : \{0, 1\}^n \rightarrow \{0, 1\}^m$, this algorithm outputs a garbled circuit GC along with a vector of 0-labels $K[y]$ on output wires.
- $M[y] \leftarrow \text{Eval}(GC, M[x])$: Given a garbled circuit GC and a vector of garbled labels $M[x]$ on input vector x , this algorithm outputs a vector of garbled labels $M[y]$ on output vector y .

For security, we assume that the garbling scheme satisfies obliviousness [BHR12]. That is, there exists a simulator $\mathcal{S}$, given a circuit $\mathcal{C}$, that can simulate a garbled circuit GC and a vector of garbled labels $M[x]$, which are computationally indistinguishable from the real values.

Functionality $\mathcal{F}_{2PC}$

This functionality initializes two identifier-value lists Bit and Val, where each value in Bit (resp., Val) is an element in $\mathbb{F}_2$ (resp., $\mathbb{F}_{2^\lambda}$). It interacts with two parties P_0 and P_1 .

Input: Upon receiving (input, id, b , x) from P_b and (input, id, b) from P_{1-b} , where $b, x \in \mathbb{F}_2$, set $\text{Bit}[\text{id}] := x$.

Eval: Upon receiving (eval, $\{\text{id}^{(x_i)}\}_{i \in [0, n]}, \{\text{id}^{(y_i)}\}_{i \in [0, m]}, \mathcal{C}$) from both parties, where $\mathcal{C} : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ is a boolean circuit, compute $(\text{Bit}[\text{id}^{(y_0)}], \text{Bit}[\text{id}^{(y_1)}], \dots, \text{Bit}[\text{id}^{(y_{m-1})}]) := \mathcal{C}(\text{Bit}[\text{id}^{(x_0)}], \text{Bit}[\text{id}^{(x_1)}], \dots, \text{Bit}[\text{id}^{(x_{n-1})}])$.

Rand: Upon receiving (rand, id) from both parties, sample $\text{Val}[\text{id}] \leftarrow \mathbb{F}_{2^\lambda}$.

Pack: Upon receiving (pack, $\{\text{id}^{(i)}\}_{i \in [0, \lambda]}, \text{id}$) from both parties, compute $\text{Val}[\text{id}] := \sum_{i \in [0, \lambda]} \text{Bit}[\text{id}^{(i)}] \cdot X^i \in \mathbb{F}_{2^\lambda}$.

Unpack: Upon receiving (unpack, id, $\{\text{id}^{(i)}\}_{i \in [0, \lambda]}$) from both parties, decompose $\text{Val}[\text{id}] := \sum_{i \in [0, \lambda]} x^{(i)} \cdot X^i \in \mathbb{F}_{2^\lambda}$ and define $\text{Bit}[\text{id}^{(i)}] := x^{(i)} \in \mathbb{F}_2$ for each $i \in [0, \lambda]$.

Open: Upon receiving (open, id) from both parties, send $\text{Val}[\text{id}] \in \mathbb{F}_{2^\lambda}$ to the adversary, wait for $x \in \mathbb{F}_{2^\lambda}$ from the adversary, and send x to both parties. If $x \neq \text{Val}[\text{id}]$, set a cheat flag.

Check: This command is allowed to be called only once. Upon receiving (check) from both parties, do the following:

1. Wait for a predicate $P : \mathbb{F}_2^{|\mathcal{I}|} \times \mathbb{F}_{2^\lambda}^{|\mathcal{J}|} \rightarrow \mathbb{F}_2$ from the adversary, where $\mathcal{I}$ (resp., $\mathcal{J}$) is the set of all available identifiers in list Bit (resp., Val).
2. If $P(\{\text{Bit}[\text{id}]\}_{\text{id} \in \mathcal{I}}, \{\text{Val}[\text{id}]\}_{\text{id} \in \mathcal{J}}) = 0$ or a cheat flag is set, abort.

Figure 3: Functionality for secure two-party computation with one-bit leakage.

4 Constant-Round 2PC with Active Security

In Figure 3, we give a 2PC functionality $\mathcal{F}_{2PC}$ in the active setting. This functionality allows two parties to input bits via the (input) command and generate random elements in $\mathbb{F}_{2^\lambda}$ via the (rand). By calling the (eval), two parties can compute any Boolean circuit. Two parties are able to call the (open) command to open some elements in $\mathbb{F}_{2^\lambda}$ to both of them. We do not consider the (output) command to output values to only one party, as it is not required for our RAM-based 2PC protocol (shown in Section 6). In addition, we define the (pack) and (unpack) commands to realize the conversion between λ bits and one element in $\mathbb{F}_{2^\lambda}$. Finally, a malicious adversary, who corrupts either P_0 or P_1 , can leak at most one-bit information on secret elements by inputting a predicate P only once.

Based on a garbling scheme and functionality $\mathcal{F}_{\text{aBit}}$, we present an efficient 2PC protocol Π_{2PC} with active security in Figure 4. This protocol adopts the dual-execution framework [MF06], and securely realizes functionality $\mathcal{F}_{2PC}$ (Figure 3). Note that the check procedure works as the batch check of SPDZ-style authenticated sharings, where BDOZ-style authenticated sharings are converted into SPDZ-style ones. The checking result allows a malicious adversary to make a selective-failure attack, i.e., an incorrect guess on the secret values will lead to the protocol aborts, and a correct guess will make the honest party accept. All the checks are done at the end of protocol execution, and thus the adversary can reveal at most one-bit information.

We use the Yao's 2PC protocol [Yao86] based on garbling schemes to securely compute any Boolean circuit, and adopt dual execution to achieve active security with one-bit leakage. In the original dual execution [MF06, HKE12], each of two parties first acts as a garbler and then acts as an evaluator, and then both parties execute an equality check immediately after the circuit was computed. Different from the original dual execution, we defer the check to the open phase, and use sub-protocol $\Pi_{\text{BatchCheck}}$ to perform the

Protocol Π_{2PC} (Part I)

This protocol invokes $\Pi_{\text{BatchCheck}}$ (Figure 2) as a sub-protocol, and adopts a garbling scheme $\mathcal{GS} = (\text{Garble}, \text{Eval})$.

Initialize: For each $b \in \mathbb{F}_2$, P_b samples $\Delta_b \leftarrow \mathbb{F}_{2^\lambda}$ such that $\text{lsb}(\Delta_b) = b$, and sends $(\text{init}, b, \Delta_b)$ to $\mathcal{F}_{\text{aBit}}$.

Input: For each $b \in \mathbb{F}_2$, for each input bit $x \in \mathbb{F}_2$ held by P_b , two parties P_0 and P_1 do the following:

1. P_b and P_{1-b} call $\mathcal{F}_{\text{aBit}}$ on respective inputs $(\text{auth}, b, x, 1)$ and $(\text{auth}, b, 1)$ to obtain respective outputs $M_b[x]$ and $K_{1-b}[x]$. Then, P_{1-b} samples $r \leftarrow \mathbb{F}_{2^\lambda}$ and send r to P_b . Next, P_{1-b} updates $K_{1-b}[x] := K_{1-b}[x] \oplus r$, and P_b updates $M_b[x] := M_b[x] \oplus r$.
2. P_b defines $K_b[0] = s$ and P_{1-b} sets $M_{1-b}[0] = s$ by letting P_b sample $s \leftarrow \mathbb{F}_{2^\lambda}$ and send s to P_{1-b} .
3. Both parties define $\langle\langle x \rangle\rangle = (\langle\langle x \rangle\rangle_b, \langle\langle x \rangle\rangle_{1-b})$, where $\langle\langle x \rangle\rangle_b := (x, K_b[0], M_b[x])$ and $\langle\langle x \rangle\rangle_{1-b} := (0, K_{1-b}[x], M_{1-b}[0])$.

Eval: To compute $(y^{(0)}, \dots, y^{(m-1)}) \leftarrow \mathcal{C}(x^{(0)}, \dots, x^{(n-1)})$, two parties P_0 and P_1 use BDOZ-style authenticated sharings $\{\langle\langle x^{(i)} \rangle\rangle\}_{i \in [0, n]}$ to compute $\{\langle\langle y^{(i)} \rangle\rangle\}_{i \in [0, m]}$ as follows, where $\langle\langle x^{(i)} \rangle\rangle_b = (x_b^{(i)}, K_b[x_{1-b}^{(i)}], M_b[x_b^{(i)}])$ for each $b \in \mathbb{F}_2$ and $i \in [0, n]$.

1. For each $b \in \mathbb{F}_2$, P_b computes $K_b[x^{(i)}] := K_b[x_{1-b}^{(i)}] \oplus x_b^{(i)} \cdot \Delta_b$ and P_{1-b} computes $M_{1-b}[x^{(i)}] := M_{1-b}[x_{1-b}^{(i)}]$ such that $M_{1-b}[x^{(i)}] = K_b[x^{(i)}] \oplus x^{(i)} \cdot \Delta_b$ for each $i \in [0, n]$.
2. As a garbler, for each $b \in \mathbb{F}_2$, P_b runs $(GC_b, \{K_b[y^{(i)}]\}_{i \in [0, m]}) \leftarrow \text{Garble}(\{K_b[x^{(i)}]\}_{i \in [0, n]}, \Delta_b, \mathcal{C})$, and sends GC_b to P_{1-b} .
3. As an evaluator, for each $b \in \mathbb{F}_2$, P_{1-b} runs $\{M_{1-b}[y^{(i)}]\}_{i \in [0, m]} \leftarrow \text{Eval}(\{M_{1-b}[x^{(i)}]\}_{i \in [0, n]}, GC_b)$.
4. For each $b \in \mathbb{F}_2$, P_b computes $y_b^{(i)} := \text{lsb}(K_b[y^{(i)}] \oplus M_b[y^{(i)}]) \in \mathbb{F}_2$ and $\langle\langle y^{(i)} \rangle\rangle_b := (y_b^{(i)}, K_b[y^{(i)}] \oplus y_b^{(i)} \cdot \Delta_b, M_b[y^{(i)}])$ for each $i \in [0, m]$. As a result, both parties hold BDOZ-style authenticated sharing $\langle\langle y^{(i)} \rangle\rangle$ for each $i \in [0, m]$.

Figure 4: Actively secure constant-round 2PC protocol with one-bit leakage in the $(\mathcal{F}_{\text{aBit}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.

verification of dual execution, where garbled labels in the dual execution are transformed into BDOZ-style authenticated sharings which are in turn converted into SPDZ-style ones.

Our 2PC protocol requires a garbling scheme (e.g., half-gates [ZRE15]) to be compatible with free XOR [KS08]. In this case, we can set the global key in authenticated sharings as the global offset in free XOR. As a result, garbled labels can be converted to BDOZ-style authenticated sharings. To obtain garbled labels to evaluate a garbled circuit, the two parties maintain the invariant that, for each wire carrying bit x , they hold a BDOZ-style authenticated sharing $\langle\langle x \rangle\rangle$. Such a sharing can be obtained from (i) calling $\mathcal{F}_{\text{aBit}}$ to authenticate an input bit, or (ii) computing it from garbled labels on the wire. Functionality $\mathcal{F}_{\text{aBit}}$ allows the corrupted party to choose its output, and thus it fails to comply with the uniform distribution of 0-labels on input wires. Thus, we randomize each 0-label with a public randomness r . The correctness of garbling scheme gives $M_{1-b}[y^{(i)}] = K_b[y^{(i)}] \oplus y^{(i)} \cdot \Delta_b$ for each $b \in \{0, 1\}$ and output bit $y^{(i)}$. From $\text{lsb}(\Delta_0 \oplus \Delta_1) = 1$, we have

$$\begin{aligned} y_b^{(i)} \oplus y_{1-b}^{(i)} &= \text{lsb}(K_b[y^{(i)}] \oplus M_b[y^{(i)}]) \oplus \text{lsb}(K_{1-b}[y^{(i)}] \\ &\quad \oplus M_{1-b}[y^{(i)}]) = y^{(i)} \cdot \text{lsb}(\Delta_b \oplus \Delta_{1-b}) = y^{(i)}. \end{aligned}$$

Reactive 2PC. For the sake of simplicity, we describe the protocol Π_{2PC} (Figure 4) to securely compute a single Boolean circuit. Nevertheless, protocol Π_{2PC} is natural to support reactive computation, as the state information can be transferred via BDOZ-style authenticated sharings, and this protocol realizes the efficient conversion between BDOZ-style authenticated sharings and garbled labels in the dual execution.

Protocol Π_{2PC} (Part II)

Rand: To compute SPDZ-style authenticated sharing $\llbracket r \rrbracket$ for a random $r \leftarrow \mathbb{F}_{2^\lambda}$, two parties P_0 and P_1 do the following:

1. For each $b \in \mathbb{F}_2$, P_b samples $\mathbf{r}_b \leftarrow \mathbb{F}_2^\lambda$, and then P_b and P_{1-b} call functionality $\mathcal{F}_{aBit}$ on respective inputs $(\text{auth}, b, \mathbf{r}_b, \lambda)$ and $(\text{auth}, b, \lambda)$ to obtain respective outputs $M_b[\mathbf{r}_b]$ and $K_{1-b}[\mathbf{r}_b]$.
2. Two parties define $\langle\langle \mathbf{r} \rangle\rangle = (\langle\langle \mathbf{r} \rangle\rangle_0, \langle\langle \mathbf{r} \rangle\rangle_1)$, where $\langle\langle \mathbf{r} \rangle\rangle_b := (\mathbf{r}_b, K_b[\mathbf{r}_{1-b}], M_b[\mathbf{r}_b])$ for each $b \in \mathbb{F}_2$, and run $\langle\langle r \rangle\rangle := \text{B2F}(\langle\langle \mathbf{r} \rangle\rangle)$ and $\llbracket r \rrbracket := \text{Convert}(\langle\langle r \rangle\rangle)$.

Pack: To pack BDOZ-style authenticated sharings $\{\langle\langle x^{(i)} \rangle\rangle\}_{i \in [0, \lambda]}$ into one SPDZ-style authenticated sharing $\llbracket x \rrbracket$ such that $x = \sum_{i \in [0, \lambda]} x^{(i)} \cdot X^i \in \mathbb{F}_{2^\lambda}$, both parties run $\langle\langle x \rangle\rangle := \text{B2F}(\langle\langle x^{(0)} \rangle\rangle, \dots, \langle\langle x^{(\lambda-1)} \rangle\rangle)$ and $\llbracket x \rrbracket := \text{Convert}(\langle\langle x \rangle\rangle)$.

Unpack: To unpack $\llbracket x \rrbracket$ into $\{\langle\langle x^{(i)} \rangle\rangle\}_{i \in [0, \lambda]}$ such that $x = \sum_{i \in [0, \lambda]} x^{(i)} \cdot X^i \in \mathbb{F}_{2^\lambda}$, two parties P_0 and P_1 do the following:

1. For each $b \in \mathbb{F}_2$, P_b decomposes $x_b \in \mathbb{F}_{2^\lambda}$ in $\llbracket x \rrbracket_b$ as $\mathbf{x}_b = (x_b^{(0)}, \dots, x_b^{(\lambda-1)}) \in \mathbb{F}_2^\lambda$ such that $x_b = \sum_{i \in [0, \lambda]} x_b^{(i)} \cdot X^i$, and then P_b and P_{1-b} call $\mathcal{F}_{aBit}$ on respective inputs $(\text{auth}, b, \mathbf{x}_b, \lambda)$ and $(\text{auth}, b, \lambda)$ to obtain respective outputs $M_b[\mathbf{x}_b]$ and $K_{1-b}[\mathbf{x}_b]$.
2. Both parties define $(\langle\langle x^{(0)} \rangle\rangle, \dots, \langle\langle x^{(\lambda-1)} \rangle\rangle) = \langle\langle \mathbf{x} \rangle\rangle := (\langle\langle \mathbf{x} \rangle\rangle_0, \langle\langle \mathbf{x} \rangle\rangle_1)$, where $\langle\langle \mathbf{x} \rangle\rangle_b := (x_b, K_b[\mathbf{x}_{1-b}], M_b[\mathbf{x}_b])$ for each $b \in \mathbb{F}_2$.
3. Both parties run $\langle\langle \tilde{x} \rangle\rangle := \text{B2F}(\langle\langle \mathbf{x} \rangle\rangle)$ and $\llbracket \tilde{x} \rrbracket := \text{Convert}(\langle\langle \tilde{x} \rangle\rangle)$.
4. Both parties locally compute $\llbracket y \rrbracket := \llbracket x \rrbracket - \llbracket \tilde{x} \rrbracket$, and run sub-protocol $\Pi_{\text{BatchCheck}}$ (Figure 2) on input $(\llbracket y \rrbracket, 0)$ to check $y = 0$.

Open: To open $x \in \mathbb{F}_{2^\lambda}$ in $\llbracket x \rrbracket$, where $\llbracket x \rrbracket_b = (x_b, M_b[x])$ for each $b \in \mathbb{F}_2$, P_0 sends $x_0 \in \mathbb{F}_{2^\lambda}$ to P_1 , and P_1 sends $x_1 \in \mathbb{F}_{2^\lambda}$ to P_0 in parallel. Two parties output $\tilde{x} := x_0 \oplus x_1$, and run sub-protocol $\Pi_{\text{BatchCheck}}$ (Figure 2) on input $(\llbracket x \rrbracket, \tilde{x})$ to check $x = \tilde{x}$.

Check: The consistency of values, sent to $\mathcal{F}_{aBit}$ or two parties, has been checked by running sub-protocol $\Pi_{\text{BatchCheck}}$. All these checks are done in a batch at the end of protocol execution.

Figure 4: Actively secure constant-round 2PC protocol with one-bit leakage in the $(\mathcal{F}_{aBit}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.

Specifically, a reactive computation consists of a series of circuits $(\mathcal{C}_0, \dots, \mathcal{C}_\ell)$, and each circuit $\mathcal{C}_j$ takes as input a state σ_{j-1} and a bit string $x_j \in \{0, 1\}^n$, and outputs an updated state σ_j and a bit string $y_j \in \{0, 1\}^m$. For each Boolean circuit $\mathcal{C}_j$, our protocol Π_{2PC} is able to take as input $\langle\langle \sigma_{j-1} \rangle\rangle$ and $\langle\langle x_j \rangle\rangle$ and then output $\langle\langle \sigma_j \rangle\rangle$ and $\langle\langle y_j \rangle\rangle$. When computing circuit $\mathcal{C}_{j+1}$, Π_{2PC} can use $\langle\langle \sigma_j \rangle\rangle$ and $\langle\langle x_{j+1} \rangle\rangle$ to compute $\langle\langle \sigma_{j+1} \rangle\rangle$ and $\langle\langle y_{j+1} \rangle\rangle$. In this way, protocol Π_{2PC} is able to securely perform the whole reactive computation.

Security. The active security of protocol Π_{2PC} is stated in Theorem 1, and we give its proof in Appendix A.

Theorem 1. *Let $\mathcal{GS}$ be a garbling scheme with obliviousness. Then, protocol Π_{2PC} (Figure 4) securely realizes functionality $\mathcal{F}_{2PC}$ (Figure 3) against malicious adversaries in the $(\mathcal{F}_{aBit}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.*

5 Actively Secure Distributed Point Function

In Figure 6, we define an ideal functionality $\mathcal{F}_{DPF}$ for distributed point function in the active setting. Similarly, it allows the adversary to make a single selective-failure query by inputting a predicate. Then, we present an actively secure two-party protocol Π_{DPF} (shown in Figure 5) to instantiate $\mathcal{F}_{DPF}$. In this protocol, we suppose that the BDOZ-style and SPDZ-style authenticated sharings input by two parties have been generated by executing the **Input** and **Pack** of protocol Π_{2PC} . Our actively secure DPF protocol builds

Protocol Π_{DPF}

This protocol invokes $\Pi_{\text{BatchCheck}}$ (Figure 2) as a sub-protocol.

Initialize: For each $b \in \mathbb{F}_2$, P_b samples $\Delta_b \leftarrow \mathbb{F}_{2^\lambda}$ such that $\text{lsb}(\Delta_b) = b$, and sends $(\text{init}, b, \Delta_b)$ to $\mathcal{F}_{\text{aBit}}$.

Protocol inputs: Two parties P_0 and P_1 hold n BDOZ-style authenticated sharings $\langle\langle \alpha^{(i)} \rangle\rangle = (\langle\langle \alpha^{(i)} \rangle\rangle_0, \langle\langle \alpha^{(i)} \rangle\rangle_1)$ for all $i \in [0, n)$ as well as a SPDZ-style authenticated sharing $\llbracket \beta \rrbracket = (\llbracket \beta \rrbracket_0, \llbracket \beta \rrbracket_1)$. Let $N = 2^n$ for some $n \in \mathbb{N}$. Let $\mathcal{H}_0 : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be a CCR hash function and $\mathcal{H}_1 : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ such that $\mathcal{H}_1(x) := \mathcal{H}_0(x) \parallel \mathcal{H}_0(x \oplus 1)$.

Generate SPDZ-style authenticated sharings of DPF outputs: Let $\langle\langle \alpha^{(i)} \rangle\rangle_b = (\alpha_b^{(i)}, \mathsf{K}_b[\alpha_{1-b}^{(i)}], \mathsf{M}_b[\alpha_b^{(i)}])$ and $\llbracket \beta \rrbracket_b = (\beta_b, \mathsf{M}_b[\beta])$ for each $b \in \{0, 1\}$. The parties P_0 and P_1 do the following.

1. Both parties call $\mathcal{F}_{\text{coin}}$ to sample a public randomness $W \in \mathbb{F}_{2^\lambda}$. Each party P_b sets $(s_b^{(0,0)} \parallel t_b^{(0,0)}) := \Delta_b \oplus W \in \{0, 1\}^\lambda$.
2. For each $b \in \{0, 1\}$, for each $i \in [0, n)$, P_b computes the following:

$$\text{CW}_b^{(i)} := \left(\bigoplus_{j \in [0, 2^i)} \mathcal{H}_0(s_b^{(i,j)} \parallel t_b^{(i,j)}) \right) \oplus \Delta_b \oplus \left(\alpha_b^{(i)} \cdot \Delta_b \oplus \mathsf{K}_b[\alpha_{1-b}^{(i)}] \oplus \mathsf{M}_b[\alpha_b^{(i)}] \right) \in \{0, 1\}^\lambda,$$

and sends $\text{CW}_b^{(i)}$ to P_{1-b} . For each $i \in [0, n)$, both parties compute $\text{CW}^{(i)} := \text{CW}_0^{(i)} \oplus \text{CW}_1^{(i)}$, and each party P_b computes:

$$\begin{aligned} (s_b^{(i+1,2j)} \parallel t_b^{(i+1,2j)}) &:= \mathcal{H}_0(s_b^{(i,j)} \parallel t_b^{(i,j)}) \oplus t_b^{(i,j)} \cdot \text{CW}^{(i)} \text{ for each } j \in [0, 2^i), \\ (s_b^{(i+1,2j+1)} \parallel t_b^{(i+1,2j+1)}) &:= \mathcal{H}_0(s_b^{(i,j)} \parallel t_b^{(i,j)}) \oplus (s_b^{(i,j)} \parallel t_b^{(i,j)}) \oplus t_b^{(i,j)} \cdot \text{CW}^{(i)} \text{ for each } j \in [0, 2^i). \end{aligned}$$

3. For each $b \in \{0, 1\}$, P_b computes

$$\text{CW}_b^{(n)} := \left(\bigoplus_{j \in [0, N)} \mathcal{H}_1(s_b^{(n,j)} \parallel t_b^{(n,j)}) \right) \oplus (\beta_b \parallel \mathsf{M}_b[\beta]) \in \{0, 1\}^{2\lambda},$$

and sends $\text{CW}_b^{(n)}$ to P_{1-b} . Then, both parties compute $\text{CW}^{(n)} := \text{CW}_0^{(n)} \oplus \text{CW}_1^{(n)}$. For each $b \in \{0, 1\}$, P_b computes

$$\begin{aligned} \llbracket u^{(j)} \rrbracket_b &:= (u_b^{(j)} = t_b^{(n,j)}, \mathsf{M}_b[u^{(j)}] = (s_b^{(n,j)} \parallel t_b^{(n,j)})) \text{ for each } j \in [0, N), \\ \llbracket v^{(j)} \rrbracket_b &:= (v_b^{(j)} \parallel \mathsf{M}_b[v^{(j)}]) := \mathcal{H}_1(s_b^{(n,j)} \parallel t_b^{(n,j)}) \oplus t_b^{(n,j)} \cdot \text{CW}^{(n)} \text{ for each } j \in [0, N). \end{aligned}$$

4. As in the **Rand** process of protocol $\Pi_{2\text{PC}}$ (Figure 4), both parties call functionality $\mathcal{F}_{\text{aBit}}$ to generate $\llbracket r \rrbracket$ with a random $r \in \mathbb{F}_{2^\lambda}$. Then, both parties call functionality $\mathcal{F}_{\text{coin}}$ to sample a random challenge $\chi \in \mathbb{F}_{2^\lambda}$, and locally compute

$$\llbracket a \rrbracket := \sum_{j \in [0, N)} \chi^j \cdot \llbracket u^{(j)} \rrbracket + \sum_{j \in [0, N)} \chi^{N+j} \cdot \llbracket v^{(j)} \rrbracket + \llbracket r \rrbracket.$$

5. As in the **Open** process of protocol $\Pi_{2\text{PC}}$, both parties open $\llbracket a \rrbracket$ to obtain $\tilde{a} = a_0 + a_1 \in \mathbb{F}_{2^\lambda}$ by letting P_0 send a_0 to P_1 and P_1 send a_1 to P_0 in parallel. Then, both parties run sub-protocol $\Pi_{\text{BatchCheck}}$ (Figure 2) on input $(\llbracket a \rrbracket, \tilde{a})$ to check $a = \tilde{a}$.

6. For each $j \in [0, N)$, both parties obtain $\llbracket u^{(j)} \rrbracket = (\llbracket u^{(j)} \rrbracket_0, \llbracket u^{(j)} \rrbracket_1)$ and $\llbracket v^{(j)} \rrbracket = (\llbracket v^{(j)} \rrbracket_0, \llbracket v^{(j)} \rrbracket_1)$.

Figure 5: Actively secure two-party protocol for DPF with one-bit leakage in the $(\mathcal{F}_{\text{aBit}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.

upon the semi-honest DPF protocol [GYW⁺23], which is based on circular correlation robust (CCR) hash functions [GKWy20, GKW⁺20].

Functionality $\mathcal{F}_{\text{DPF}}$

This functionality initializes two identifier-value lists Bit and Val, where each value in Bit (resp., Val) is an element in $\mathbb{F}_2$ (resp., $\mathbb{F}_{2^\lambda}$). It interacts with two parties P_0 and P_1 .

Input: For $b \in \{0, 1\}$, upon receiving (input, id, b , x) from P_b and (input, id, b) from P_{1-b} , where either $x \in \mathbb{F}_2$ or $x \in \mathbb{F}_{2^\lambda}$, set either $\text{Bit}[\text{id}] := x$ or $\text{Val}[\text{id}] := x$ depending on whether x is a bit or not.

Gen: Upon receiving (gen, $\{\text{id}^{(\alpha_i)}\}_{i \in [0, n]}$, id, $\{\text{id}^{(i)}\}_{i \in [0, 2N]}$) from P_0 and P_1 where $N = 2^n$, do the following:

1. Compute $\alpha := \sum_{i \in [0, n]} \text{Bit}[\text{id}^{(\alpha_i)}] \cdot 2^i \in [0, N)$ and set $\beta := \text{Val}[\text{id}] \in \mathbb{F}_{2^\lambda}$.
2. Perform the following:

$$\begin{aligned} (\text{Val}[\text{id}^{(0)}], \dots, \text{Val}[\text{id}^{(N-1)}]) &:= \mathbf{unit}(N, \alpha) \in \mathbb{F}_{2^\lambda}^N, \\ (\text{Val}[\text{id}^{(N)}], \dots, \text{Val}[\text{id}^{(2N-1)}]) &:= \mathbf{unit}(N, \alpha) \cdot \beta \in \mathbb{F}_{2^\lambda}^N. \end{aligned}$$

Check: This command is allowed to be called only once. Upon receiving (check) from both parties, do the following:

1. Wait for a predicate $P : \mathbb{F}_2^{|\mathcal{I}|} \times \mathbb{F}_{2^\lambda}^{|\mathcal{J}|} \rightarrow \mathbb{F}_2$ from the adversary, where $\mathcal{I}$ (resp., $\mathcal{J}$) is the set of all available identifiers in list Bit (resp., Val).
2. If $P(\{\text{Bit}[\text{id}]\}_{\text{id} \in \mathcal{I}}, \{\text{Val}[\text{id}]\}_{\text{id} \in \mathcal{J}}) = 0$, abort.

Figure 6: Functionality for DPF with one-bit leakage.

Definition 2. Let $\mathcal{H} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$, χ be a distribution on $\{0, 1\}^\lambda$, $\mathcal{F}_{\lambda+1, \lambda}$ be a family of functions with $(\lambda+1)$ -bit input and λ -bit output, and $\mathcal{O}_{\mathcal{H}, \Delta}^{\text{ccr}}(x, b) := \mathcal{H}(x \oplus \Delta) \oplus b \cdot \Delta$ be an oracle for $x, \Delta \in \{0, 1\}^\lambda$ and $b \in \{0, 1\}$.

We say that $\mathcal{H}$ is (t, q, ρ, ϵ) -CCR if for any distinguisher $\mathcal{D}$ running in time at most t and making at most q queries to $\mathcal{O}_{\mathcal{H}, \Delta}^{\text{ccr}}(\cdot, \cdot)$, and any χ with min-entropy at least ρ , it holds

$$\left| \Pr_{\Delta \leftarrow \chi} \left[\mathcal{D}^{\mathcal{O}_{\mathcal{H}, \Delta}^{\text{ccr}}(\cdot, \cdot)}(1^\lambda) = 1 \right] - \Pr_{f \leftarrow \mathcal{F}_{\lambda+1, \lambda}} \left[\mathcal{D}^{f(\cdot, \cdot)}(1^\lambda) = 1 \right] \right|$$

is at most ϵ , where $\mathcal{D}$ cannot query both $(x, 0)$ and $(x, 1)$ for any $x \in \{0, 1\}^\lambda$.

Compared to the prior semi-honest DPF protocol [GYW⁺23], our actively secure protocol Π_{DPF} performs a consistency check on all leaf nodes. If a corrupted party sends an incorrect share of a correction word and makes a wrong guess on some prefix of α to remove this error, then the error will propagate in the tree expansion of Π_{DPF} and fail the check. Allowing the adversary to guess a prefix of α leads to one-bit leakage.

Through a simple induction, protocol Π_{DPF} ensures that, for $i \in [0, n]$ and $j \in [0, 2^i)$,

$$(s_b^{(i, j)} \parallel t_b^{(i, j)}) \oplus (s_{1-b}^{(i, j)} \parallel t_{1-b}^{(i, j)}) = \begin{cases} 0, j \neq \alpha^{(0)}, \dots, \alpha^{(i-1)} \\ \Delta_b \oplus \Delta_{1-b}, \text{ otherwise} \end{cases}$$

As $\text{lsb}(\Delta_0 \oplus \Delta_1) = 1$, one can check that $\llbracket \mathbf{u} \rrbracket$ is a vector of SDPZ-style authenticated sharings on $\mathbf{u} = \mathbf{unit}(N, \alpha)$. Moreover, given the above equality, $v^{(j)} := v_b^{(j)} \oplus v_{1-b}^{(j)} = \beta \in \mathbb{F}_{2^\lambda}$ and $\text{M}_b[v^{(j)}] \oplus \text{M}_{1-b}[v^{(j)}] = \text{M}_b[\beta] \oplus \text{M}_{1-b}[\beta]$ if and only if $j = \alpha$. Thus, $\llbracket \mathbf{v} \rrbracket$ is a vector of SPDZ-style authenticated sharings on $\mathbf{v} = \mathbf{unit}(N, \alpha) \cdot \beta$.

Security. We state the security of our DPF protocol Π_{DPF} in Theorem 2, and provide its proof in Appendix B.

Functionality $\mathcal{F}_{\text{RAM2PC}}$

This functionality initialize an identifier-value array Bit , where each entry in Bit is an element in $\mathbb{F}_2$. It interacts with two parties P_0 and P_1 .

Initialize: Upon receiving (init, N) from both parties where $N = 2^n$, initialize a memory list $\mathbf{D} := (D^{(0)}, \dots, D^{(N-1)}) = (0, \dots, 0) \in \mathbb{F}_{2^\lambda}^N$.

Input: For $b \in \{0, 1\}$, upon receiving $(\text{input}, \text{id}, b, x)$ from P_b and $(\text{input}, \text{id}, b)$ from P_{1-b} with $x \in \mathbb{F}_2$, set $\text{Bit}[\text{id}] := x$.

Private RAM read/write access: Upon receiving $(\text{access}, F, \{\text{id}^{(\alpha, i)}\}_{i \in [0, n]}, \{\text{id}^{(\text{aux}, i)}\}_{i \in [0, \ell]}, \{\text{id}^{(\text{aux}', i)}\}_{i \in [0, \ell]})$ from P_0 and P_1 , where $\ell \in \mathbb{N}$ and $F : \{0, 1\}^\lambda \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\lambda \times \{0, 1\}^\ell$ is a Boolean circuit, do the following:

1. Compute $\alpha := \sum_{i \in [0, n]} \text{Bit}[\text{id}^{(\alpha, i)}] \cdot 2^i \in [0, N)$ and $\text{aux} := (\text{Bit}[\text{id}^{(\text{aux}, 0)}], \dots, \text{Bit}[\text{id}^{(\text{aux}, \ell-1)}]) \in \{0, 1\}^\ell$.
2. Compute $(D'^{(\alpha)}, \text{aux}') := F(D^{(\alpha)}, \text{aux})$.
3. Update $D^{(\alpha)} := D'^{(\alpha)}$.
4. Set $(\text{Bit}[\text{id}^{(\text{aux}', 0)}], \dots, \text{Bit}[\text{id}^{(\text{aux}', \ell-1)}]) := \text{aux}'$.

Check: This command is allowed to be called only once. Upon receiving (check) from both parties, do the following:

1. Wait for a predicate $P : \mathbb{F}_2^{|I|} \times \mathbb{F}_{2^\lambda}^N \rightarrow \mathbb{F}_2$ from the adversary, where I is the set of all identifiers in Bit .
2. If $P(\{\text{Bit}[\text{id}]\}_{\text{id} \in I}, \mathbf{D}) = 0$, abort.

Figure 7: Functionality for RAM-based 2PC with one-bit leakage.

Theorem 2. Let $\mathcal{H}_0$ be a CCR hash function. Then, protocol Π_{DPPF} (Figure 5) securely realizes functionality $\mathcal{F}_{\text{DPPF}}$ (Figure 6) against malicious adversaries in the $(\mathcal{F}_{\text{aBit}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.

6 RAM-based 2PC with Active Security

We present our RAM-based two-party computation functionality $\mathcal{F}_{\text{RAM2PC}}$ in Figure 7, along with its instantiation Π_{RAM2PC} in Figure 8. As discussed in Section 2, we follow the blueprint in Floram [Ds17], which was designed for the semi-honest setting. We use a Read-Only Memory (ROM), a Write-Only Memory (WOM), a refresh procedure synchronizing these two types of memory, and a linear-scan stash to store updates between two refresh procedures. For readers who are familiar with Floram, we note that the main difference is in the structure of ROM and WOM, which now needs to store authenticated shares to prevent active attacks.

ROM and WOM structure. Consider an N -element memory $\mathbf{D}$. In our protocol, two parties construct a WOM $\mathbf{W}$ from $\mathbf{D}$ with each holding $\llbracket W^{(i)} \rrbracket := \llbracket D^{(i)} \rrbracket$ for every $i \in [0, N)$. They also build a ROM $\mathbf{R}$ where each has the same value $R^{(i)} := (D^{(i)} \parallel D^{(i)} \cdot \Delta) \oplus F(k_0, i) \oplus F(k_1, i)$ for every $i \in [0, N)$, with $k_0, k_1 \in \mathbb{F}_{2^\lambda}$ held by P_0 and P_1 respectively. Here, $F : \mathbb{F}_{2^\lambda} \times [0, N) \rightarrow \mathbb{F}_{2^\lambda}^2$ is a PRF.

To realize a read operation on position α from ROM, two parties input $\langle \alpha \rangle$ and a dummy $\llbracket \beta \rrbracket$ (e.g., $\llbracket \beta \rrbracket = \llbracket 0 \rrbracket$) to Π_{DPPF} . Then, $\llbracket D^{(\alpha)} \rrbracket$ can be computed from using inner product $\langle \mathbf{R}, \{[u^{(i)}]\}_{i \in [0, N)} \rangle$ to select its masked entry $[R^{(\alpha)}]$ and removing mask $[F(k_0, \alpha) \oplus F(k_1, \alpha)]$, which is computed using 2PC.

To implement a write operation from WOM such that $\llbracket D^{(\alpha)} \rrbracket$ is updated to $\llbracket D^{(\alpha)} \rrbracket \oplus \epsilon$, two parties input $\langle \alpha \rangle$ and a random $\llbracket \beta \rrbracket$ to Π_{DPPF} to obtain $\llbracket \mathbf{u} \rrbracket$ and $\llbracket \mathbf{v} \rrbracket$. Then, they open $\llbracket \epsilon \oplus \beta \rrbracket$ to obtain $\epsilon \oplus \beta$ and compute the difference $\llbracket \delta \rrbracket = \llbracket \mathbf{u} \rrbracket \cdot (\epsilon \oplus \beta) \oplus \llbracket \mathbf{v} \rrbracket$ with $\delta = \text{unit}(N, \alpha) \cdot \epsilon \in \mathbb{F}_{2^\lambda}^N$. Two parties update WOM $\llbracket \mathbf{W} \rrbracket := \llbracket \mathbf{W} \rrbracket \oplus \llbracket \delta \rrbracket$.

Protocol Π_{RAM2PC}

This protocol invokes $\Pi_{2\text{PC}}$ (Figure 4) and Π_{DPF} (Figure 5) as two sub-protocols, and maintains three memories: the ROM R , WOM $\llbracket W \rrbracket$ and stash $\langle\langle S \rangle\rangle$. Let $\text{PRF} : \{0, 1\}^\lambda \times [0, N) \rightarrow \{0, 1\}^{2\lambda}$ be a pseudo-random function (PRF) and $\sigma \in \mathbb{N}$ denote the maximum number of entries in a stash.

Initialize: Two parties P_0 and P_1 execute **Initialize** of sub-protocol $\Pi_{2\text{PC}}$ to initialize two global keys $\Delta_0 \in \mathbb{F}_{2^\lambda}$ and $\Delta_1 \in \mathbb{F}_{2^\lambda}$. Then, both parties execute as follows:

1. Both parties set $\llbracket W^{(i)} \rrbracket := \llbracket 0 \rrbracket$ for each $i \in [0, N)$ to initialize WOM $\llbracket W \rrbracket$, and initialize stash $\langle\langle S \rangle\rangle := \emptyset$.
2. Both parties run the following **Refresh** procedure to initialize ROM $R \in \mathbb{F}_{2^\lambda}^N$.
3. For an initial auxiliary input aux , both parties execute **Input** of sub-protocol $\Pi_{2\text{PC}}$ to generate $\langle\langle \text{aux} \rangle\rangle$.

Full private access: To obliviously read or write an entry in the α -th position with $\alpha \in [0, N)$ and $N = 2^n$, P_0 and P_1 hold $\langle\langle \alpha \rangle\rangle = (\langle\langle \alpha^{(0)} \rangle\rangle, \dots, \langle\langle \alpha^{(n-1)} \rangle\rangle)$ such that $\alpha^{(i)} \in \{0, 1\}$ for $i \in [0, n)$ and $\sum_{i \in [0, n)} \alpha^{(i)} \cdot 2^i = \alpha$, and then do the following:

1. Both parties execute **Rand** of sub-protocol $\Pi_{2\text{PC}}$ to generate $\llbracket \beta \rrbracket$ with a random element $\beta \in \mathbb{F}_{2^\lambda}$.
2. Both parties execute sub-protocol Π_{DPF} on the input $\{\langle\langle \alpha^{(i)} \rangle\rangle\}_{i \in [0, n)}$ and $\llbracket \beta \rrbracket$ to obtain $\llbracket u \rrbracket$ and $\llbracket v \rrbracket$ such that u (resp., v) is an unit vector with exactly one nonzero entry $u^{(\alpha)} = 1$ (resp., $v^{(\alpha)} = \beta$).
3. Both parties locally compute a pair of unauthenticated additive sharings $([c], [d]) := \sum_{i \in [0, N)} (R^{(i)}[0], R^{(i)}[1]) \cdot [u^{(i)}]$, where $R^{(i)} = (R^{(i)}[0], R^{(i)}[1]) \in \{0, 1\}^{2\lambda}$ for $i \in [0, N)$, and $[u^{(i)}]$ for all $i \in [0, N)$ are the additive secret sharings defined in $\llbracket u \rrbracket$.
4. P_0 and P_1 execute **Eval** of sub-protocol $\Pi_{2\text{PC}}$ on the input $(\langle\langle k_0 \rangle\rangle, \langle\langle k_1 \rangle\rangle, \langle\langle \alpha \rangle\rangle, \langle\langle \text{aux} \rangle\rangle)$ to perform the following computation:
 - (a) If there exists an entry $(\langle\langle \alpha \rangle\rangle, \langle\langle x \rangle\rangle)$ in $\langle\langle S \rangle\rangle$, set $\langle\langle y \rangle\rangle := \langle\langle x \rangle\rangle$. Otherwise, compute $([y], [y \cdot \Delta]) := \text{PRF}(\langle\langle k_0 \rangle\rangle, \langle\langle \alpha \rangle\rangle) \oplus \text{PRF}(\langle\langle k_1 \rangle\rangle, \langle\langle \alpha \rangle\rangle) \oplus ([c]_0, [d]_0) \oplus ([c]_1, [d]_1)$, set $\llbracket y \rrbracket = ([y], [y \cdot \Delta])$ and unpack $\llbracket y \rrbracket$ as $\langle\langle y \rangle\rangle$. (Note that both parties can run **Unpack** of sub-protocol $\Pi_{2\text{PC}}$ on the input $\llbracket y \rrbracket$ to generate $\langle\langle y \rangle\rangle$.)
 - (b) Compute $(\langle\langle y' \rangle\rangle, \langle\langle \text{aux}' \rangle\rangle) := F(\langle\langle y \rangle\rangle, \langle\langle \text{aux} \rangle\rangle)$.
 - (c) If there exists an entry $(\langle\langle \alpha \rangle\rangle, \langle\langle x \rangle\rangle)$ in $\langle\langle S \rangle\rangle$, set the entry as $(\perp, \perp)$ and add $(\langle\langle \alpha \rangle\rangle, \langle\langle y' \rangle\rangle)$ to $\langle\langle S \rangle\rangle$.
5. Both parties update $\langle\langle \text{aux} \rangle\rangle$ as $\langle\langle \text{aux}' \rangle\rangle$, and run **Pack** of sub-protocol $\Pi_{2\text{PC}}$ on the input $\langle\langle y \rangle\rangle$ to generate $\llbracket y \rrbracket$.
6. P_0 and P_1 run $\llbracket y' \rrbracket := \text{Convert}(\langle\langle y' \rangle\rangle)$, and then locally compute $\llbracket \delta \rrbracket := \llbracket \beta \rrbracket \oplus \llbracket y \rrbracket \oplus \llbracket y' \rrbracket$. Then, both parties execute **Open** of sub-protocol $\Pi_{2\text{PC}}$ open $\llbracket \delta \rrbracket$ to obtain $\delta \in \{0, 1\}^\lambda$.
7. For $i \in [0, N)$, both parties locally compute and update $\llbracket W^{(i)} \rrbracket := \llbracket W^{(i)} \rrbracket \oplus \delta \cdot \llbracket u^{(i)} \rrbracket \oplus \llbracket v^{(i)} \rrbracket$, meaning that $\llbracket W \rrbracket$ is updated.
8. If the number of entries in stash $\langle\langle S \rangle\rangle$ is identical to σ , both parties run the following **Refresh** procedure.

Refresh: Both parties clear the stash, i.e., set $\langle\langle S \rangle\rangle := \emptyset$, and then do the following:

1. For each $b \in \{0, 1\}$, P_b samples $k_b = (k_b^{(0)}, \dots, k_b^{(\lambda-1)}) \leftarrow \{0, 1\}^\lambda$, which is used as a PRF key, and both parties execute **Input** of sub-protocol $\Pi_{2\text{PC}}$ on the input bits $k_b^{(i)}$ for $i \in [0, \lambda)$ to generate $\langle\langle k_b \rangle\rangle$.
2. For each $b \in \{0, 1\}$, for each $i \in [0, N)$, P_b computes $R_b^{(i)} = (R_b^{(i)}[0], R_b^{(i)}[1]) := \llbracket W^{(i)} \rrbracket_b \oplus \text{PRF}(k_b, i) \in \{0, 1\}^{2\lambda}$ and sends $R_b^{(i)}$ to P_{1-b} .
3. Both parties compute R by setting $R^{(i)} := R_0^{(i)} \oplus R_1^{(i)} \in \{0, 1\}^{2\lambda}$ for each $i \in [0, N)$, and also obtain $\langle\langle k_0 \rangle\rangle$ and $\langle\langle k_1 \rangle\rangle$.

Figure 8: Actively secure protocol for RAM-based 2PC with one-bit leakage.

Stash-based lookup. After a write operation, the data in ROM will no longer be current. we implement a linear-scan stash, $\langle\langle S \rangle\rangle$, in secure computation with maximum size σ . It is a temporary storage for all WOM updates that have not yet been applied to ROM. Each element in $\langle\langle S \rangle\rangle$ includes BDOZ-style authenticated sharings of an index and the updated value at this index. The two parties use $\langle\langle S \rangle\rangle$ with our ROM and WOM structures as follows:

- For a read operation, the two parties also search for a valid value in $\langle\langle S \rangle\rangle$ with the index they intend to read. If found, this value, rather than the value from ROM, will returned as the output.
- For a write operation, the two parties clear all values in $\langle\langle S \rangle\rangle$ with the same index in the current operation. Then, they append this new updated value to $\langle\langle S \rangle\rangle$.

Refresh procedure. Every write operation updates the authenticated shares in WOM to reflect the most recent content. However, as the stash grows, the cost to access it will grow; thus we need a refresh procedure to update the content of ROM so that $\langle\langle S \rangle\rangle$ can be emptied. In this procedure, each party P_b samples a secret key k_b to mask their authenticated shares for every $i \in [0, N)$ to obtain $R_b^{(i)} := (W_b^{(i)} \parallel M_b[W^{(i)}]) \oplus F(k_b, i)$. Then, it sends this masked value to P_{1-b} and computes $\mathbf{R} := \mathbf{R}_b \oplus \mathbf{R}_{1-b}$. Finally, each party inputs its secret key k_b to the secure computation to allow read operations. Note that a refresh procedure requires no secure computation due to the ROM and WOM structure.

Similar to Floram, a refresh procedure is invoked after σ write operations, and the stash-based lookup incurs an $O(\sigma)$ overhead for both read and write operations. So, setting σ to $O(\sqrt{N})$ can achieve the best in the overall complexity.

Full private access. Similar to Floram, protocol Π_{RAM2PC} considers full private access to RAM. A full private access refers to the functionality that, on input an oblivious function F , an element $D^{(\alpha)}$ in the memory, and auxiliary input aux, update $(D^{(\alpha)}, \text{aux}) := F(D^{(\alpha)}, \text{aux})$.

We follow the blueprint of Floram to implement full private accesses from our ROM and WOM structure and stash. More specifically, the two parties do:

1. Perform a read operation to retrieve $D^{(\alpha)}$.
2. Run 2PC protocol to compute $F(D^{(\alpha)}, \text{aux})$.
3. Perform a write operation to update $D^{(\alpha)}$.

Optimization on read operations. In read operations of our protocol, we only utilize $\{[u^{(i)}]\}_{i \in [0, N)}$ from Π_{DPF} , which is independent of β . We use a technique called *tree-trimming optimization* [BGI16], to avoid the expansion of last $\log \lambda$ levels of the tree in our DPF protocol and set $\beta = 2^{(\alpha \bmod \lambda)}$. We note that $\llbracket \beta \rrbracket$ can be computed from secure computation, and the bit decomposition of the only non-zero position in $\{[v^{(i)}]\}_{i \in [0, N/2^\lambda)}$ corresponds to that in the above utilized $\{[u^{(i)}]\}_{i \in [0, N)}$. This optimization significantly improves the efficiency of read operations for a large RAM size.

Achieving overall one-bit leakage. Note that our RAM-based 2PC protocol Π_{RAM2PC} calls the interfaces of $\Pi_{2\text{PC}}$ and Π_{DPF} , each of which invokes a consistency check that leads to 1-bit leakage therein. Since the two checks follow the same form (i.e., calling sub-protocol $\Pi_{\text{BatchCheck}}$), they can be merged at the end of protocol Π_{RAM2PC} . Intuitively, this merged consistency check is performed only once so that the adversary can only learn a one-bit predicate of all inputs of the honest party and intermediate results from whether the check passes or not. Meanwhile, all intermediate transcripts exchanged by the two parties are indistinguishable from truly random values.

Security. We present our main theorem in Theorem 3. A critical aspect of our protocol Π_{RAM2PC} is its *non-black-box* utilization of authenticated secret sharings generated in our DPF protocol Π_{DPF} . Thus, it will

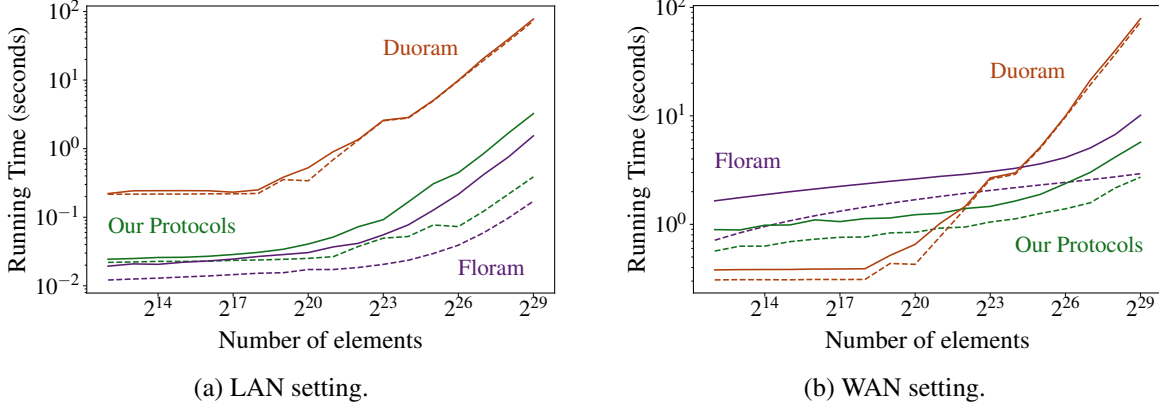


Figure 9: **Wall-clock time of an access operation using our protocol, Floram and Duoram in LAN and WAN settings.** Solid lines are for full-access operations, which support both read and write operations; dashed lines are for read-only operations. Each entry in the array has 8 bytes. All timings are average of a sufficiently large number of accesses.

invoke Π_{DPF} directly instead of $\mathcal{F}_{\text{DPF}}$ in a hybrid model. We provide a sketched proof of this theorem in Appendix C.

Theorem 3. *Let $\mathcal{H}_0$ be a CCR hash function and $\mathcal{GS}$ be a garbling scheme whose obliviousness can be based on CCR $\mathcal{H}_0$. Then, protocol Π_{RAM2PC} (Figure 8) securely realizes functionality $\mathcal{F}_{\text{RAM2PC}}$ (Figure 7) against malicious adversaries in the $(\mathcal{F}_{\text{aBit}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.*

7 Evaluation

We would like to study the performance of our protocol in the following four aspects.

- Q1** What is the cost of our actively secure protocol compared to the state-of-the-art semi-honest ones?
- Q2** How many improvements in efficiency are there when comparing our protocol to state-of-the-art maliciously secure ones?
- Q3** What is the bottleneck of our protocol in different scenarios and array sizes?
- Q4** What is the practical performance when putting our protocol in end-to-end applications?

To answer these questions, we implement our protocol and made code available in EMP [WMK16]. Below, we provide implementation details and setup, with the answers to all questions.

7.1 Experimental Setup

We implement all of our protocol in C++ based on EMP toolkit [WMK16]. We instantiate $\mathcal{F}_{\text{COT}}$ using Ferret OT [YWL⁺20] and instantiate PRFs using AES-128. All code is compiled using `gcc` version 11.4.0, with `-O3` optimization flag enabled.

Our benchmark is performed on a pair of AWS `R5.8xlarge` instances, each with 32 vCPUs and 256 GB memory. To simulate a LAN network, we use two instances in the same availability zone, and manually limit the network bandwidth to 2 Gbps; the round-trip time (RTT) between two instances is roughly 0.1 ms. To simulate a WAN network, we limit the bandwidth to 100 Mbps and set RTT to 60 ms using `tc` command. These settings are similar to related prior works [VHG23].

If not specified otherwise, we vary the number of elements in the array from 2^{12} to 2^{29} , and use an element size of 8 bytes, corresponding to up to 4 GiB of data. Read-only operations refer to reading an

element in the array without any modification, and full-access operations support both reading an element from the array and writing a new value back. Our implementation of the dual-execution-based 2PC is always single-threaded; other parts of our protocol are multi-threaded when possible. We use 16 threads by default. We set the stash size σ of our protocol to $\sqrt{N/T}/20$ in the LAN setting, and $\sigma := \sqrt{N}/16$ in the WAN setting, where N is the size of RAM and T is the number of threads.

7.2 Overhead Compared to Semi-Honest Protocols

Floram [Ds17] and Duoram [VHG23] are the state-of-the-art RAM-based 2PC protocols secure against semi-honest adversaries in the LAN and WAN settings, respectively. To understand the overhead of our protocol, we compare our cost with the cost of Floram and Duoram in both network settings and on both read-only and full-access operations. Since some protocols need cost amortization, we run a sufficient number of accesses and report the average wall-clock time across all operations.

We show the result in Figure 9. In the LAN setting, Floram is approximately $2\times$ faster than our protocol on both read-only and full-access operations. On the other hand, our protocol is about 1 – 2 orders of magnitude faster than Duoram on both operations. In the WAN setting, our protocol, interestingly, is roughly $2\times$ faster than Floram on full-access operations and has an advantage on read-only operations. Compared with Duoram, it is slower when the array size is less than 2^{21} elements. Regarding the impact of access operation types, full-access operations require about a 50% extra cost compared to read-only operations in the WAN setting and incur a five-fold overhead in the LAN setting in our protocol and Floram. For Duoram, their read-only and full access have similar performance.

Our protocol only imposes a constant overhead on top of Floram; thus its performance is similar to Floram’s. In particular, we need roughly $2\times$ overhead in both secure computation and local expansion of trees needed in DPF. At the same time, since our underlying malicious secure DPF protocol integrates optimizations shown in Half-Tree [GYW⁺23], our protocol shows nearly no overhead for active security compared to Floram. When compared to Duoram, our protocol needs $O(\log N)$ rounds for an access operation to an array of size N , but Duoram has an amortized constant roundtrips by performing an offline phase for $\log N$ operations together. As a result, our protocol performs worse than Duoram when the array size is small in the WAN setting. However, when the array size is sufficiently large or in the LAN setting, our protocol still outperforms Duoram because computation is the bottleneck, not the roundtrip. Regardless, one can conclude that our active protocol is competitive with state-of-the-art semi-honest protocols.

7.3 Comparison to SOTA Active Protocols

The state-of-the-art maliciously secure protocol is KY18 [KY18]. As mentioned before, this protocol uses a generic compilation from malicious MPC [LPSY15, LSS16, KOS16] and ORAM protocol [WCS15] to RAM-based MPC. Their actively secure MPC is instantiated by a SPDZ protocol with an offline and an online phase. We contacted the authors and utilized their script to obtain an accurate estimation of the cost of KY18. We first calculate the number of field multiplication triples for an access operation from the number of AND gates required; then we estimate the wall-clock time based on the state-of-the-art triple generation protocol [KPR18] instead of MASCOT used in KY18 for the most up-to-date estimation. We also compare against a possible combination by using the authenticated garbling [WRK17a] (dubbed KY18-AGC) instead of SPDZ-BMR. KY18 noted that such a combination might not be compatible with their memory optimizations and put it an open problem; nevertheless, it represents a hypothetical best possible solution. Since KY18 has the same complexity on read and write operations, we compare one full-access operation of our protocol (which supports both), and one write operation of KY18 and KY18-AGC. Note that the results for KY18 and KY18-AGC are depicted for array sizes ranging from 2^9 to 2^{24} 8-byte elements; this is because KY18 only provides the number of AND gates required for these array sizes.

We report the comparison result in Figure 10 and can observe that our protocol consistently outperforms KY18 by about two orders of magnitude in both network settings. It is also nearly one order of magnitude

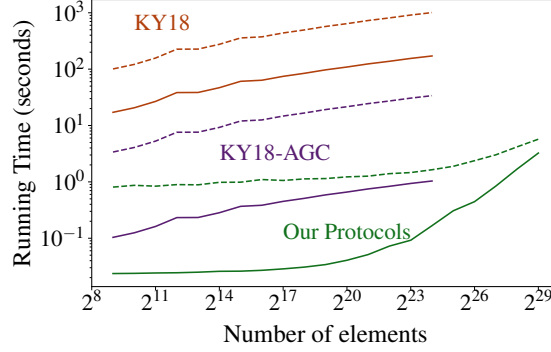


Figure 10: **Wall-clock time of a full-access operation using our protocol, KY18 and KY18-AGC in both network settings.** Solid lines are in LAN settings; dashed lines are in WAN settings.

faster than the hypothetical result KY18-AGC. However, it’s important to mention that our protocol has a one-bit leakage to the overall protocol (not each access operation); thus it presents a trade-off between security and efficiency. Additionally, we observe a trend that the performance gap of our protocol between LAN and WAN settings narrows as the array size increases due to the increasing cost of computation. This does not happen for KY18, as their computation complexity is also sublinear. So, for the commonly used array size in MPC applications, our protocol provides a much better trade-off and enables efficient access with just one-bit leakage.

7.4 Microbenchmarks

We delve into our protocol and analyze the cost of each part of an access operation. We divide the cost of an access operation into six steps: 1) DPF generation; 2) memory access to read secret shares from public encrypted ROM; 3) secure evaluation of PRFs; 4) secure linear scan of stash; 5) memory access to write authenticated secret shares to WOM; 6) refresh cost amortized over each full-access operation. Note that a full-access operation has all these 6 steps while a read-only operation only has the first 4 steps.

We record the average wall-clock time of each part for a full-access operation as well as a read-only operation for array of size 2^{24} , 2^{26} , and 2^{28} in different network settings.

Figure 11a presents the cost breakdown of an access operation in different scenarios with LAN settings. In a full-access operation, the cost of securely computing PRF remains constant, but other costs increase as the array size increases. Notably, the cost of computing PRF becomes the bottleneck with small array sizes, but it is minor when the size is sufficiently large. Refresh and stash scanning costs take an insignificant portion of the total costs across all scenarios. Conversely, the costs of DPF generation and memory access increase approximately linearly with the array size, becoming the primary expense when the array size is sufficiently large. Applying parallelization can efficiently mitigate it, and our protocol with 16 threads performs approximately twice as fast as using only one thread by utilizing multithreading in local computation. We also notice the bottleneck in a read-only operation differs from that in a full-access operation, where scanning ROM memory rather than DPF generation becomes the primary cost in read-only operations since our protocol requires a relatively small DPF tree with tree-trimming optimization in Section 6.

In WAN settings, as illustrated in Figure 11b, the cost distribution significantly differs. Local memory access takes a minor fraction of the total cost of an access operation, thus multithreading has a minor effect on mitigating the overall costs. Meanwhile, 2PC (including PRF and stash scanning) along with the refresh procedure incur a considerably higher cost due to bandwidth limitations, and become the bottlenecks of an access operation for large array sizes. While DPF generation remains the principal cost factor, unlike in LAN settings, it does not increase linearly with the array size. This is because the latency from round complexity contributes significantly to the cost since there is an $O(\log N)$ round complexity on DPF generation.

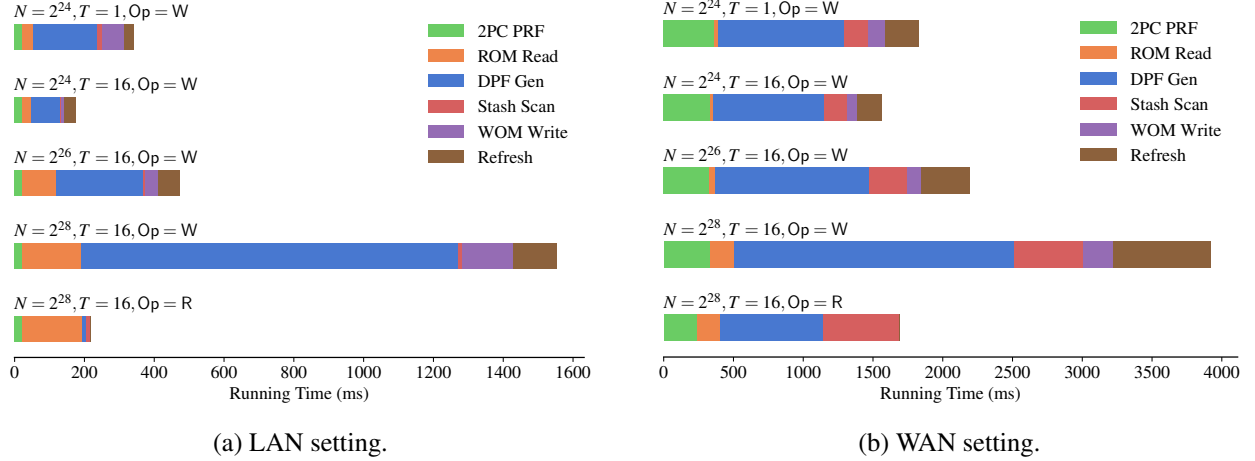


Figure 11: **Cost breakdown of an access operation in different scenarios.** N is the size of an array and T is the maximum number of threads used. $Op=R$ denotes a read-only operation, and $Op=W$ denotes a full-access operation. Elements in the array have a size of 8 bytes. The wall-clock time of each part is averaged from a number of access operations, which is multiple of the refresh period.

Compared to a full-access operation, DPF generation in a read-only operation is faster since roughly $\log \lambda$ rounds are eliminated from tree-trimming optimization.

7.5 RAM Applications

To benchmark the performance of our protocol in real-world scenarios, we extend our protocol to several RAM applications: oblivious binary search, stable matching and the script function. We report all the results in Table 1 and present our experiments in detail below.

Benchmark	Parameters	LAN (sec)	WAN (sec)
Binary Search	1 search	81.68	185.2
	2^5 searches	120.42	1118.7
	2^{10} searches	1894.23	30989.6
Gale-Shapley	2^3 pairs	9.2	268.7
	2^6 pairs	670.3	19975.0
	2^9 pairs	44476.1	<i>about 19 days</i>
Script	$N = 2^5, r = 8$	42.1	1159.1
	$N = 2^{10}, r = 1$	167.6	4721.4
	$N = 2^{10}, r = 8$	1396.4	41022.7

Table 1: **Summary of benchmark results.** All results are wall-clock time in seconds if not specified. The array contains 2^{25} 8-byte elements for all binary search benchmarks.

Binary search. RAM-based MPC protocols can obviously execute binary searches on an array with a few access operations. The performance of such searches in an array has been benchmarked in various protocols [GKK⁺12, ZWR⁺16, Ds17].

We extend our protocol and implement an oblivious binary search. It needs $O(\log N)$ read-only operations for each search on an array of N elements. To evaluate it, we set the array size of 2^{25} 8-byte elements, and record the wall-clock time of executing 1, 2^5 , and 2^{10} searches on both network settings. This time is

composed of initialization of RAM structure and performing read-only operations.

Stable matching. Gale-Shapley algorithm [GS62] is a typical solution to the stable matching problem. We extend our RAM-based 2PC protocol to implement an oblivious version of the Gale-Shapley algorithm, aimed at benchmarking performance in a complex, end-to-end application.

Our implementation closely follows the origin Gale-Shapley algorithm, and it requires $O(n^2)$ access operations of arrays size of up to n^2 elements for matching n pairs.

We evaluate the wall-clock time for full protocol execution, including tests with 8, 64 and 512 pairs in LAN settings and 8 and 64 pairs in WAN settings. We also estimate the wall-clock time for 512 pairs in WAN settings based on the microbenchmark results mentioned above. We notice since the array size is not sufficiently large, securely computing PRF is the primary overhead for access operations. Consequently, our protocol shows a relatively poor performance when the number of pairs is small, but becomes efficient as the pair count increases. Doerner et al. [DEs16] proposed several optimized algorithms for stable matching, all of which can be implemented using the building blocks proposed in this paper as well.

Script. Script is a key derivation function intended to provide resistance against parallelized brute-force attacks by using a large amount of memory. We implement an oblivious script function to enable securely executing some cryptographic functions using RAM-based 2PC.

We denote the cost factor of a script function by N , the parallelization factor by p , and the block size factor by r . Our implementation requires $O(Nr)$ read-only operations of an array size of Nr 1Kbit elements for computing a function. We select three representative parameters [Per09] and benchmark each of them in both network settings.

8 Future Work

Castro and Polychroniadou [dCP22] proposed a malicious DPF protocol in a weaker model with two servers and one client where at most one party can be corrupted. It is a future work to apply our optimizations to their model. It would also be interesting to apply this protocol to improve the concrete efficiency of client correlation generation.

9 Acknowledgements

Work of Kang Yang is supported by the National Key Research and Development Program of China (Grant No. 2022YFB2702000), and by the National Natural Science Foundation of China (Grant Nos. 62102037 and 61932019). Work of Yu Yu is supported by the National Natural Science Foundation of China (Grant Nos. 62125204 and 61872236). Yu Yu's work has also been supported by the New Cornerstone Science Foundation through the XPLOER PRIZE. Work of Xiao Wang is supported by NSF awards #2236819 and #2310927.

References

- [AFN⁺17] Ittai Abraham, Christopher W. Fletcher, Kartik Nayak, Benny Pinkas, and Ling Ren. Asymptotically tight bounds for composing ORAM with PIR. In Serge Fehr, editor, *PKC 2017, Part I*, volume 10174 of *LNCS*, pages 91–120, Amsterdam, The Netherlands, March 28–31, 2017. Springer, Heidelberg, Germany.
- [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 291–308, London, UK, November 11–15, 2019. ACM Press.

- [BCG⁺20] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 387–416, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Heidelberg, Germany.
- [BCG⁺22] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Nicolas Resch, and Peter Scholl. Correlated pseudorandomness from expand-accumulate codes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 603–633, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Heidelberg, Germany.
- [BDOZ11] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 169–188, Tallinn, Estonia, May 15–19, 2011. Springer, Heidelberg, Germany.
- [BEE⁺17] Johes Bater, Gregory Elliott, Craig Eggen, Satyender Goel, Abel Kho, and Jennie Rogers. Smcql: Secure querying for federated databases. *Proc. VLDB Endow.*, page 673–684, feb 2017.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.
- [BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796, Raleigh, NC, USA, October 16–18, 2012. ACM Press.
- [BLN⁺21] Sai Sheshank Burra, Enrique Larraia, Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, Emmanuela Orsini, Peter Scholl, and Nigel P. Smart. High-performance multi-party computation for binary circuits based on oblivious transfer. *Journal of Cryptology*, 34(3):34, July 2021.
- [Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.
- [CWYY23] Hongrui Cui, Xiao Wang, Kang Yang, and Yu Yu. Actively secure half-gates with minimum overhead under duplex networks. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 35–67, Lyon, France, April 23–27, 2023. Springer, Heidelberg, Germany.
- [dCP22] Leo de Castro and Antigoni Polychroniadou. Lightweight, maliciously secure verifiable function secret sharing. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 150–179, Trondheim, Norway, May 30 – June 3, 2022. Springer, Heidelberg, Germany.
- [DEs16] Jack Doerner, David Evans, and abhi shelat. Secure stable matching at scale. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1602–1613, Vienna, Austria, October 24–28, 2016. ACM Press.

- [DKL⁺13] Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In Jason Crampton, Sushil Jajodia, and Keith Mayes, editors, *ESORICS 2013*, volume 8134 of *LNCS*, pages 1–18, Egham, UK, September 9–13, 2013. Springer, Heidelberg, Germany.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany.
- [Ds17] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 523–535, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [FJKW15] Sky Faber, Stanislaw Jarecki, Sotirios Kentros, and Boyang Wei. Three-party ORAM for secure computation. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 360–385, Auckland, New Zealand, November 30 – December 3, 2015. Springer, Heidelberg, Germany.
- [GGH⁺13] Craig Gentry, Kenny A. Goldman, Shai Halevi, Charanjit S. Jutla, Mariana Raykova, and Daniel Wichs. Optimizing ORAM and using it efficiently for secure computation. In Emiliano De Cristofaro and Matthew K. Wright, editors, *PETS 2013*, volume 7981 of *LNCS*, pages 1–18, Bloomington, IN, USA, July 10–12, 2013. Springer, Heidelberg, Germany.
- [GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658, Copenhagen, Denmark, May 11–15, 2014. Springer, Heidelberg, Germany.
- [GKK⁺12] S. Dov Gordon, Jonathan Katz, Vladimir Kolesnikov, Fernando Krell, Tal Malkin, Mariana Raykova, and Yevgeniy Vahlis. Secure two-party computation in sublinear (amortized) time. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 513–524, Raleigh, NC, USA, October 16–18, 2012. ACM Press.
- [GKW18] S. Dov Gordon, Jonathan Katz, and Xiao Wang. Simple and efficient two-server ORAM. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 141–157, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Heidelberg, Germany.
- [GKW⁺20] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 793–822, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Heidelberg, Germany.
- [GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press.
- [GO96] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.

- [Gol04] Oded Goldreich. *Foundations of Cryptography: Basic Applications*, volume 2. Cambridge University Press, Cambridge, UK, 2004.
- [GS62] David Gale and Lloyd Stowell Shapley. College admissions and the stability of marriage. *American Mathematical Monthly*, 1962.
- [GYW⁺23] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 330–362, Lyon, France, April 23–27, 2023. Springer, Heidelberg, Germany.
- [HKE12] Yan Huang, Jonathan Katz, and David Evans. Quid-Pro-Quo-tocols: Strengthening semi-honest protocols with dual execution. In *2012 IEEE Symposium on Security and Privacy*, pages 272–284, San Francisco, CA, USA, May 21–23, 2012. IEEE Computer Society Press.
- [HSS17] Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 598–628, Hong Kong, China, December 3–7, 2017. Springer, Heidelberg, Germany.
- [HV21] Ariel Hamlin and Mayank Varia. Two-server distributed ORAM with sublinear computation and constant rounds. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 499–527, Virtual Event, May 10–13, 2021. Springer, Heidelberg, Germany.
- [KOS15] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Actively secure OT extension with optimal overhead. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 724–741, Santa Barbara, CA, USA, August 16–20, 2015. Springer, Heidelberg, Germany.
- [KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 830–842, Vienna, Austria, October 24–28, 2016. ACM Press.
- [KPR18] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 158–189, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Heidelberg, Germany.
- [KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498, Reykjavik, Iceland, July 7–11, 2008. Springer, Heidelberg, Germany.
- [KS14] Marcel Keller and Peter Scholl. Efficient, oblivious data structures for MPC. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 506–525, Kaoshiung, Taiwan, R.O.C., December 7–11, 2014. Springer, Heidelberg, Germany.

- [KY18] Marcel Keller and Avishay Yanai. Efficient maliciously secure multiparty computation for RAM. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 91–124, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Heidelberg, Germany.
- [LO13] Steve Lu and Rafail Ostrovsky. Distributed oblivious RAM for secure two-party computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 377–396, Tokyo, Japan, March 3–6, 2013. Springer, Heidelberg, Germany.
- [LPSY15] Yehuda Lindell, Benny Pinkas, Nigel P. Smart, and Avishay Yanai. Efficient constant round multi-party computation combining BMR and SPDZ. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 319–338, Santa Barbara, CA, USA, August 16–20, 2015. Springer, Heidelberg, Germany.
- [LSS16] Yehuda Lindell, Nigel P. Smart, and Eduardo Soria-Vazquez. More efficient constant-round multi-party computation from BMR and SHE. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part I*, volume 9985 of *LNCS*, pages 554–581, Beijing, China, October 31 – November 3, 2016. Springer, Heidelberg, Germany.
- [LWN⁺15] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. OblivM: A programming framework for secure computation. In *2015 IEEE Symposium on Security and Privacy*, pages 359–376, San Jose, CA, USA, May 17–21, 2015. IEEE Computer Society Press.
- [MF06] Payman Mohassel and Matthew Franklin. Efficiency tradeoffs for malicious two-party computation. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 458–473, New York, NY, USA, April 24–26, 2006. Springer, Heidelberg, Germany.
- [Per09] Colin Percival. Stronger key derivation via sequential memory-hard functions, 2009.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Heidelberg, Germany.
- [RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 16–20, 2021. Springer, Heidelberg, Germany.
- [VHG23] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A bandwidth-efficient distributed ORAM for 2- and 3-party computation. In *USENIX Security 2023*, pages 3907–3924, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.
- [WCS15] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. Circuit ORAM: On tightness of the Goldreich-Ostrovsky lower bound. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 850–861, Denver, CO, USA, October 12–16, 2015. ACM Press.

- [WHC⁺14] Xiao Shaun Wang, Yan Huang, T.-H. Hubert Chan, abhi shelat, and Elaine Shi. SCORAM: Oblivious RAM for secure computation. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *ACM CCS 2014*, pages 191–202, Scottsdale, AZ, USA, November 3–7, 2014. ACM Press.
- [WMK16] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [WRK17a] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Authenticated garbling and efficient maliciously secure two-party computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 21–37, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [WRK17b] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Global-scale secure multiparty computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 39–56, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [WYKW21] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press.
- [YWL⁺20] Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated OT with small communication. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1607–1626, Virtual Event, USA, November 9–13, 2020. ACM Press.
- [YWZ20] Kang Yang, Xiao Wang, and Jiang Zhang. More efficient MPC from improved triple generation and authenticated garbling. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1627–1646, Virtual Event, USA, November 9–13, 2020. ACM Press.
- [ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250, Sofia, Bulgaria, April 26–30, 2015. Springer, Heidelberg, Germany.
- [ZWR⁺16] Samee Zahur, Xiao Shaun Wang, Mariana Raykova, Adria Gascón, Jack Doerner, David Evans, and Jonathan Katz. Revisiting square-root ORAM: Efficient random access in multiparty computation. In *2016 IEEE Symposium on Security and Privacy*, pages 218–234, San Jose, CA, USA, May 22–26, 2016. IEEE Computer Society Press.

A Proof of Theorem 1

Theorem 1. *Let $\mathcal{GS}$ be a garbling scheme with obliviousness. Then, protocol Π_{2PC} (Figure 4) securely realizes functionality $\mathcal{F}_{2PC}$ (Figure 3) against malicious adversaries in the $(\mathcal{F}_{aBit}, \mathcal{F}_{coin}, \mathcal{F}_{com})$ -hybrid model.*

Proof (Sketch). As the two parties are symmetric in Π_{2PC} , we w.l.o.g. assume that P_b is corrupted and P_{1-b} is honest for some fixed $b \in \{0, 1\}$. Simulator $\mathcal{S}_{2PC}$ extracts $\Delta_b \in \mathbb{F}_{2^\lambda}$ from emulated $\mathcal{F}_{aBit}$ (and aborts if $\text{lsb}(\Delta_b) \neq b$) and maintains the secret share of corrupted P_b for each BDOZ- or SPDZ-style authenticated sharing.

In particular, maintaining such shares is straightforward for all input bits (in **Input**), random values (in **Rand**) and unpacked bits (in **Unpack**) since $\mathcal{S}_{2PC}$ emulates $\mathcal{F}_{aBit}$. In **Eval**, $\mathcal{S}_{2PC}$ sends a uniformly random GC_{1-b} (which is indistinguishable from a real one due to obliviousness) to corrupted P_b , and uses Δ_b and the extracted secret shares of P_b to evaluate garbled circuit GC_{1-b} to compute P_b 's secret shares of the BDOZ-style authenticated sharings of circuit outputs. As **Pack** only includes local computation, $\mathcal{S}_{2PC}$ follows the same computation to maintain shares.

Given the extracted secret shares of corrupted P_b , $\mathcal{S}_{2PC}$ works as follows. In **Input**, $\mathcal{S}_{2PC}$ extracts all input bits of corrupted P_b from emulated $\mathcal{F}_{aBit}$ and sends them to $\mathcal{F}_{2PC}$. In **Open**, $\mathcal{S}_{2PC}$ receives $x \in \mathbb{F}_{2^\lambda}$ from $\mathcal{F}_{2PC}$ and sends $x_{1-b} := x \oplus x_b$ to P_b , where x_b is given by the extracted P_b 's secret share of SPDZ-style authenticated sharing $\llbracket x \rrbracket$. In **Check**, $\mathcal{S}_{2PC}$ emulates $\mathcal{F}_{coin}$ to get $\chi \in \mathbb{F}_{2^\lambda}$ and extracts $V_b \in \mathbb{F}_{2^\lambda}$ from emulated $\mathcal{F}_{com}$. Moreover, $\mathcal{S}_{2PC}$ constructs a predicate (i.e., a mixed circuit) P such that, on input all values authenticated in BDOZ or SPDZ style in Π_{2PC} (or equivalently, all values stored in $\mathcal{F}_{2PC}$), it uses these inputs, the extracted inputs and secret shares of P_b , and the public random coins to define $V_{1-b} \in \mathbb{F}_{2^\lambda}$ in an equivalent way and outputs 1 if and only if $V_{1-b} = V_b$.

The two worlds are indistinguishable unless the adversary breaks the obliviousness of garbling schemes or leads to a non-negligible difference in abort probability. Note that the latter difference is negligible given uniform χ . $\square$

B Proof of Theorem 2

Theorem 2. *Let $\mathcal{H}_0$ be a CCR hash function. Then, protocol Π_{DPF} (Figure 5) securely realizes functionality $\mathcal{F}_{DPF}$ (Figure 6) against malicious adversaries in the $(\mathcal{F}_{aBit}, \mathcal{F}_{coin}, \mathcal{F}_{com})$ -hybrid model.*

Proof (Sketch). As the two parties are symmetric in Π_{DPF} , we w.l.o.g. assume that P_b is corrupted and P_{1-b} is honest for some fixed $b \in \{0, 1\}$. Simulator $\mathcal{S}_{DPF}$ extracts $\Delta_b \in \mathbb{F}_{2^\lambda}$ from emulated $\mathcal{F}_{aBit}$ (and aborts if $\text{lsb}(\Delta_b) \neq b$) and maintains all secret shares held by corrupted P_b . These shares can be computed from public randomness W and the extracted Δ_b , each $\langle \alpha^{(i)} \rangle_b$, and $\llbracket \beta \rrbracket_b$.

$\mathcal{S}_{DPF}$ sends a uniformly sampled $CW_{1-b}^{(i)}$ to corrupted P_b for each $i \in [0, n]$. This is indistinguishable from the real-world execution, where $CW_{1-b}^{(i)} = CW^{(i)} \oplus CW_b^{(i)}$, since $CW^{(i)}$ is pseudorandom given a CCR hash function $\mathcal{H}_0$ and the entropy of Δ_{1-b} . $\mathcal{S}_{DPF}$ also receives $CW_b^{(i)'} from corrupted P_b to extract additive noise $\delta^{(i)} := CW_b^{(i)'} \oplus CW_b^{(i)}$ for each $i \in [0, n]$.$

To simulate batch check, $\mathcal{S}_{DPF}$ sends a uniformly random a_{1-b} to corrupted P_b . Then, it constructs a predicate P such that, on input all values authenticated in BDOZ or SPDZ style in Π_{DPF} (or equivalently, all values stored in $\mathcal{F}_{DPF}$), it uses a prefix $\alpha^{(0)}, \dots, \alpha^{(\ell-1)}$ for some maximal $\ell \in [0, n]$ such that $\delta^{(\ell)} \neq 0$, the extracted secret shares of corrupted P_b , and the public random coins to (i) remove additive noises on the prefix path per level, and (ii) follow the protocol specification of P_b to compute $V_b' \in \mathbb{F}_{2^\lambda}$. This predicate outputs 1 if and only if $V_b' = V_b$, which is extracted from emulated $\mathcal{F}_{com}$.

The two worlds are indistinguishable unless the adversary breaks the CCR security of $\mathcal{H}_0$ or leads to a non-negligible difference in abort probability. Note that the latter difference is negligible given uniform χ . $\square$

C Proof of Theorem 3

Theorem 3. *Let $\mathcal{H}_0$ be a CCR hash function and $\mathcal{GS}$ be a garbling scheme whose obliviousness can be based on CCR $\mathcal{H}_0$. Then, protocol Π_{RAM2PC} (Figure 8) securely realizes functionality $\mathcal{F}_{\text{RAM2PC}}$ (Figure 7) against malicious adversaries in the $(\mathcal{F}_{\text{aBit}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{com}})$ -hybrid model.*

Proof (Sketch). Since the two parties are symmetric in Π_{RAM2PC} , we w.l.o.g. assume that P_b is corrupted and P_{1-b} is honest for some fixed $b \in \{0, 1\}$. Note that protocol Π_{RAM2PC} invokes two sub-protocols $\Pi_{2\text{PC}}$ and Π_{DPF} as subroutines. Simulator $\mathcal{S}_{\text{RAM2PC}}$ can invoke simulator $\mathcal{S}_{2\text{PC}}$ and $\mathcal{S}_{\text{DPF}}$ to simulate the transcripts of the two sub-protocols. A subtle issue is that, since $\Pi_{2\text{PC}}$ and Π_{DPF} share the same global authentication keys Δ_0 and Δ_1 , the indistinguishability between these transcripts and truly random values sampled in $\mathcal{S}_{\text{RAM2PC}}$ (as per invoked $\mathcal{S}_{2\text{PC}}$ and $\mathcal{S}_{\text{DPF}}$) cannot be reduced to the obliviousness of garbling schemes or the CCR security of $\mathcal{H}_0$ in a black-box way. However, we note that the obliviousness of garbling schemes can also be based on CCR [ZRE15, RR21]. So, we can use a CCR-based garbling scheme in a *non-black-box* way and prove the above indistinguishability using CCR.

Except the transcripts sent in the two sub-protocols, the only additional transcript exchanged between the two parties is in **Refresh** procedure. This transcript is indistinguishable from a truly random value, which is sampled by $\mathcal{S}_{\text{RAM2PC}}$, due to the PRF security.

To incur abort in the ideal world with nearly the same probability in the real world, $\mathcal{S}_{\text{RAM2PC}}$ constructs a predicate P in **Check** such that it “stacks” the predicates in $\mathcal{S}_{2\text{PC}}$ and $\mathcal{S}_{\text{DPF}}$ according to how the real execution of Π_{RAM2PC} invokes the commands in $\Pi_{2\text{PC}}$ and Π_{DPF} . The abort probability can have negligible difference in the two worlds due the uniformness of χ . $\square$